Универзитет у Крагујевцу

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Крагујевац, 2026.

Универзитет у Крагујевцу



Назив студијског програма: ВЕШТАЧКА ИНТЕЛИГЕНЦИЈА

Ниво студија: Мастер академске студије

Предмет: Напредно машинско учење

Број индекса: 1ВИ/2023

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Комисија за преглед и одбрану:

1. др Александар Пеулић, редовни професор
2. др Јасна Радуловић, редовни професор
3. др Милан Чабаркапа, доцент
4. др Иван Крстић, ванредни професор
5. ванр. проф. др Владимир М. Миловановић - ментор

Датум одбране: _____

Оцена: _____

У оквиру овог мастер рада кандидат треба да истражи могућност прогнозирања временски-променљивих међутржишних зависности између криптовалута и конвенционалних финансијских актива применом модела машинског учења. У склопу тога кандидат треба да:

1. прикупи историјске дневне цене за Bitcoin и традиционалне активе (акцијске индексе, фондове племенитих метала, индекс америчког долара, Ethereum) и изврши одговарајућу предобраду и синхронизацију скупа података;
2. развије репродуцибилан рачунарски поступак за израчунавање временски-променљивих корелационих циљева помоћу клизних прозора различитих дужина и конструкцију предиктивних особина заснованих на инерцији зависности, волатилности и повратима;
3. реализује ванузорачну евалуацију и поређење неколико модела машинског учења (AR, ElasticNet, Ridge, Random Forest, GBM, XGBoost) са DCC-GARCH(1,1) економетријским репером применом процедуре проклизавајућег прозора и Diebold-Mariano теста статистичке значајности;
4. уведе практични сигнални слој за рано детектовање стресних дана на тржишту традиционалних актива на основу динамике криптовалута и прогноза зависности, и вреднује га метрикама класификационе тачности прикладним за неуравнотежен скуп класа.

Препоручена литература:

[1] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models", у: Journal of Business & Economic Statistics 20.3 (2002.), стр. 339–350.

[2] Francis X. Diebold и Roberto S. Mariano, „Comparing Predictive Accuracy", у: Journal of Business & Economic Statistics 13.3 (1995.), стр. 253–263.

[3] Hui Zou и Trevor Hastie, „Regularization and Variable Selection via the Elastic Net", у: Journal of the Royal Statistical Society: Series B 67.2 (2005.), стр. 301–320.

[4] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine", у: The Annals of Statistics 29.5 (2001.), стр. 1189–1232.

[5] Tianqi Chen и Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System", у: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016., стр. 785–794.

Крагујевац, 3. 3. 2026.

Ментор:

Др Владимир М. Миловановић,
ванредни професор

_____

Abstract

This paper examines whether the dynamics of cryptocurrency markets provide insights into how their correlation with traditional assets changes over time, and whether this information can serve as an early warning signal for investors. Bitcoin is used as the primary cryptocurrency, and Ethereum as the secondary benchmark cryptocurrency. The set of traditional assets includes stock market indices, precious metals ETFs, and the U.S. Dollar Index.

A reproducible machine learning pipeline was developed to collect data, generate results, calculate moving correlation metrics, create functions, and evaluate models using a step-by-step procedure in comparison with the DCC-GARCH test. The results show that intermarket dependence can be predicted based on a sample for each asset pair and interval length. However, most of this predictability is due to the constancy of the dependence series itself.

Crest regression, the AR(1) model, and the HAR model yield the best results and exhibit nearly identical accuracy, as all three models capitalize on this constancy. Crest regression yields the lowest average RMSE of 0.0656. All of these models significantly outperform the DCC-GARCH benchmark. The signal for second-stage investors was also tested using cryptocurrency features and autoregressive forecasts. The signal performs modestly and is generally above the probability threshold, but it is not strong enough to independently determine market timing.

Keywords: cryptocurrencies, Bitcoin, intermarket dependency, correlation, machine learning, DCC-GARCH, forecasting, risk management.

Резиме

Тема овог рада је испитивање да ли динамика тржишта криптовалута пружа информације о томе како се њихова зависност од традиционалних финансијских актива мења током времена и да ли се те информације могу користити као сигнал раног упозорења за инвеститоре. Bitcoin се користи као основна криптовалута, а Ethereum као секундарна референтна криптовалута. Скуп традиционалних актива обухвата берзанске индексе, ETF фондове везане за племените метале и индекс америчког долара.

Развијен је поновљиви процес машинског учења за прикупљање података, конструкцију приноса, израчунавање покретних корелација, креирање карактеристика и евалуацију модела коришћењем walk-forward поступка у поређењу са DCC-GARCH репером. Резултати показују да се међутржишна зависност може предвиђати ван узорка за сваки пар актива и сваку дужину прозора. Међутим, највећи део ове предвидљивости потиче од постојаности саме серије зависности.

Ridge регресија, AR(1) модел и HAR модел постижу најбоље резултате и показују готово идентичну тачност, јер сва три модела користе ову постојаност. Ridge регресија постиже најнижи просечни RMSE од 0.0656. Сви ови модели значајно надмашују DCC-GARCH репер. Такође је тестиран сигнал за инвеститоре другог степена, заснован на карактеристикама криптовалута и ауторегресивним прогнозама. Сигнал показује умерене резултате и углавном је изнад нивоа случајности, али није довољно јак да самостално одређује тренутак уласка или изласка са тржишта.

Кључне речи: криптовалуте, Bitcoin, међутржишна зависност, корелација, машинско учење, DCC-GARCH, прогнозирање, управљање ризиком.

Contents

List of Abbreviations **8**

1 Introduction **9**
1.1 Motivation and research context . . . 9
1.2 Research questions . . . 9
1.3 Contributions of the thesis . . . 10
1.4 Thesis outline . . . 10

2 Theoretical Background and Literature Review **11**
2.1 Why intermarket dependency matters . . . 11
2.2 Cryptocurrencies and conventional assets . . . 11
2.3 Rolling correlation as a forecasting target . . . 12
2.4 Econometric benchmark: DCC-GARCH . . . 13
2.5 Machine-learning approaches to forecasting . . . 13
2.5.1 The HAR model and persistence-based forecasting . . . 13
2.5.2 Regularised linear models . . . 14
2.5.3 Tree-based ensemble models . . . 14
2.6 Walk-forward evaluation and information leakage . . . 15
2.7 Model comparison and the Diebold–Mariano test . . . 15
2.8 From dependency forecasting to investor signaling . . . 16
2.9 Positioning of the present thesis . . . 16

3 Methodology **17**
3.1 Research design . . . 17
3.2 Asset universe and data source . . . 17
3.3 Return construction and preprocessing . . . 18
3.4 Target construction: rolling dependency . . . 18
3.5 Feature engineering . . . 19
3.6 Forecasting models . . . 21
3.6.1 Baseline and autoregressive models . . . 21
3.6.2 Regularised linear models . . . 21
3.6.3 Tree ensembles and adaptive ensemble . . . 21
3.6.4 DCC-GARCH benchmark . . . 22
3.7 Investor signal layer . . . 22
3.8 Walk-forward estimation framework . . . 23
3.9 Evaluation metrics . . . 23
3.10 Model comparison methodology . . . 24
3.11 Implementation and reproducibility . . . 24
3.12 Methodological expectations . . . 24

4 Empirical Results **26**
4.1 Overview of the empirical output . . . 26
4.2 Average dependency-forecasting performance . . . 26
4.3 Representative forecast example . . . 27
4.4 Forecasting performance by asset class . . . 28
4.5 The persistence dominance result . . . 29
4.6 Feature-group ablation . . . 29

4.7 Comparison with DCC-GARCH . . . 31
4.8 Error-profile diagnostics . . . 32
4.9 Investor signal layer: average results . . . 32
4.10 Best signal configurations . . . 33
4.11 Practical interpretation of signal-layer performance . . . 34
4.12 Sensitivity to rolling window length . . . 35
4.13 Summary of empirical findings . . . 35

5 Discussion **37**
5.1 The core finding . . . 37
5.2 Why simple models dominate . . . 37
5.3 ML compared with DCC-GARCH: a fairer comparison . . . 37
5.4 Strengths and limitations of the investor signal . . . 38
5.5 Methodological choices that mattered . . . 39
5.6 Statements this thesis does not make . . . 39
5.7 Iterative model development: what changed across versions . . . 39
5.8 Limitations . . . 41

6 Conclusion **42**

Appendices

A Data Description and Additional Background **44**
A.1 The descriptive role of the asset universe . . . 44
A.2 Visual overview of the dataset . . . 44
A.3 Dataset validation and quality diagnostics . . . 46
A.4 Fisher-z transformation of the correlation target . . . 48
A.5 Why multiple rolling windows are used . . . 49
A.6 Additional implementation remarks . . . 49
A.7 Summary . . . 49

B Supplementary Forecasting Results **50**
B.1 Model-comparison figures . . . 50
B.2 Hyperparameter and feature diagnostics . . . 51
B.3 Diagnostic plots for the representative equity pair . . . 52
B.4 Sensitivity by asset pair . . . 53
B.5 Interpretation . . . 54

C Supplementary Signal-Layer Results **55**
C.1 Representative signal output . . . 55
C.2 Cross-market warning interpretation . . . 55
C.3 Summary table . . . 56

D Reproducibility and Code Structure **57**
D.1 Project organisation . . . 57
D.2 Main implementation modules . . . 57
D.3 Software environment . . . 58
D.4 Analysis notebooks . . . 58
D.5 Execution flow . . . 59
D.6 Configuration . . . 61

D.7 Verification performed during the project cleanup 61
D.8 How to rebuild the thesis artifacts 61
D.9 Final reproducibility note 62

E Additional Statistical Tests **63**
E.1 Mincer-Zarnowitz Forecast Efficiency Test 63
E.2 Residual Diagnostics: Ljung-Box and ARCH LM Tests 64
E.3 Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test 65
E.4 Automated Validation Suite 66

F Market-Event Case Studies **68**

References **74**

List of Figures

1 Rolling dependency between Bitcoin and each of the five conventional assets plus Ethereum (six pairs in total) across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series exhibits the strong time-variation and persistence that motivate the forecasting framework. 19
2 Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information. . . 28
3 Diebold–Mariano diagnostic comparing Ridge (best overall RMSE across all models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate a larger positive loss-differential statistic; they do not establish formal significance because the reported standard errors use zero HAC lags. 31
4 Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment at the 30-day correlation window ($w = 30$); the error is smoothed over a trailing 60-day window. The machine-learning model shown is Ridge, which has the lowest average RMSE overall (Table 3), compared against the DCC-GARCH benchmark. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process. 32
5 Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events. 34
6 Normalised prices of the assets included in the study. 45
7 Rolling volatility across the asset universe. 45
8 Static correlation heatmap of the return series. 46
9 Observation coverage window per ticker. All assets share the aligned date range after the ETH-USD availability cutoff in November 2017. 47
10 Log-return series with extreme observations ($|z| > 5\sigma$) highlighted in red. Crypto assets contain occasional large outliers, while conventional assets are more contained but are not free of extreme observations. 48
11 Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and reduces boundary compression. 49
12 RMSE by model and window for the representative Bitcoin–S&P 500 pair. 50
13 Model R^2 comparison for the 30-day window. 50
14 Improvement relative to the naive baseline. 50
15 Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task. 51
16 Regularisation path for the Ridge-based forecasting specification. 51
17 Random Forest feature importances for the Bitcoin–S&P 500 pair at the 30-day window. 51
18 Window sensitivity of the XGBoost-based forecasting layer. 52
19 Forecast scatter diagnostic for the Bitcoin–S&P 500 dependency experiment. 52
20 Forecast error distribution for the representative Bitcoin–S&P 500 dependency experiment. 52
21 Sensitivity analysis for the Bitcoin–NASDAQ pair. 53

22 Sensitivity analysis for the Bitcoin–gold pair. 53
23 Sensitivity analysis for the Bitcoin–silver pair. 53
24 Sensitivity analysis for the Bitcoin–UUP pair. 54
25 Sensitivity analysis for the Bitcoin–Ethereum pair. 54
26 Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window. The upper panel shows the predicted stress probability and the decision threshold; the lower panel marks flagged and missed stress events. . . 55
27 Master timeline 2018–2026: Bitcoin price, conventional-asset prices, and the 30-day rolling BTC–S&P 500 correlation, with all eleven landmark events marked. Red dotted lines = crash/shock events; green = rallies and all-time highs; blue = structural divergences; purple = institutional milestones. 68
28 Correlation regime map. Left: post-event BTC correlation (30-day average) with each conventional asset. Right: change in correlation relative to the 30-day pre-event average. Red/blue = strong positive/negative shifts. . 69
29 Event 1 — COVID Black Thursday, 12 March 2020. Source: Reuters. . . . 70
30 Event 2 — FTX collapse, 8–11 November 2022. Source: BBC News. 70
31 Event 3 — SVB banking crisis, 10 March 2023. Source: The New York Times. 71
32 Event 4 — “Liberation Day” tariff shock, 2 April 2025. Source: Reuters. . 72

List of Tables

1 Asset universe used in the thesis. 17
2 Main groups of predictors used in the dependency-forecasting layer. 20
3 Average out-of-sample forecasting performance across all dependency experiments. 26
4 Feature-group ablation, Bitcoin–S&P 500 at $w = 30$. Each row adds a block of features to the row above and keeps everything before it; # is the resulting number of columns in the design matrix. 30
5 Average investor-signal performance by classifier family, aggregated across all experiments. 33
6 Average next-day return of the conventional asset on days flagged by the logistic signal versus days left clear, by asset pair (Logit, averaged over the four windows). Returns are in basis points per day; a positive difference means flagged days are, on average, worse. The flag rate is the share of days on which the signal is active. 35
7 Data quality summary per ticker 46
8 Balanced accuracy of the Logit signal layer by asset pair and rolling-window length. Family-level averages across all pairs and windows are reported in Table 5; this breakdown shows where the signal is informative. 56
9 Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments not rejected by the conventional OLS F diagnostic ($p \geq 0.05$) and average coefficient estimates. 63
10 Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. Conventional OLS F diagnostic for H_0: $\alpha = 0, \beta = 1$; p_{MZ} is not HAC-robust. 64
11 Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$. 65
12 HLN-corrected DM diagnostic: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); $\mathrm{DM}_{\mathrm{HLN}}$: corrected statistic ($t_{n-1}$). Reported p-values retain the zero-lag variance assumption and are descriptive. 66
13 Automated validation suite, summarised by test class. All 81 tests pass on the present outputs, with none skipped. The complete per-test log, including parametrised cases and per-test timings, is written to `outputs/results/test_results.xml` in the repository. 67

List of Abbreviations

Abbreviation	Full form
AR(1)	First-order autoregressive model
AUC	Area under the ROC curve
BTC	Bitcoin (`BTC-USD`)
CSV	Comma-separated values
DCC	Dynamic Conditional Correlation
DCC-GARCH	Dynamic Conditional Correlation GARCH(1,1)
DM	Diebold–Mariano (test)
EDA	Exploratory Data Analysis
ElasticNet	Elastic Net regularised regression
ETF	Exchange-Traded Fund
ETH	Ethereum (`ETH-USD`)
F1	F1 score (harmonic mean of precision and recall)
GARCH	Generalised Autoregressive Conditional Heteroskedasticity
GBM	Gradient Boosting Machine
GLD	SPDR Gold Shares ETF (`GLD`)
GPU	Graphics Processing Unit
GSPC	S&P 500 index (`^GSPC`)
HAR	Heterogeneous Autoregressive (model)
IXIC	NASDAQ Composite index (`^IXIC`)
MAE	Mean Absolute Error
ML	Machine Learning
NASDAQ	National Association of Securities Dealers Automated Quotations
OOS	Out-of-sample
RF	Random Forest
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
S&P 500	Standard & Poor's 500 index
SLV	iShares Silver Trust ETF (`SLV`)
UUP	Invesco DB US Dollar Index Bullish Fund (`UUP`)
XGB	XGBoost (Extreme Gradient Boosting)

1 Introduction

1.1 Motivation and research context

Satoshi Nakamoto published the Bitcoin concept in 2008. And on January 5, 2009, the first "genesis block" was generated, and thus the first Bitcoin network came into operation, involving computers to generate Bitcoin blocks. Since then, the technology has evolved from an experiment to a class of high-risk assets. As a result, the cryptocurrency market has now grown so large that it exerts a real influence on the market for conventional assets, as many investors—as well as government funds—use Bitcoin both as a vehicle for storing capital and for conducting financial transactions. This creates a large pool of financial capital, the rise and fall of which can influence conventional, traditional markets. Consequently, the task arises of managing risks and identifying correlations between Bitcoin and traditional markets.

This thesis explores time-varying intermarket dependency. The time-varying statistical relationship between Bitcoin and traditional asset markets: 1) stock indices; 2) gold; 3) silver; 4) the dollar. Is such a correlation even predictable? Does the cryptocurrency market contain signals about the dynamics of traditional asset prices? To assess market risks and stress, the direction of the market is important, since the cryptocurrency market is more volatile; because of this, it is possible to predict the nature of the movement of traditional asset markets more quickly.

Let's call the investor who needs a tool for precise yield forecasting a "risk manager." A risk manager must be able to identify the moment when the nature of the markets' joint movement changes—the moment of transition from a closely correlated to a decorrelated state, and vice versa. When the likelihood of a stressful day in traditional asset markets increases—given the scale of investments—even a small signal can be very useful, allowing the Risk Manager to withdraw a portion of assets before the markets fall.

1.2 Research questions

This thesis is structured around two interrelated levels. 1) The forecasting problem: Is it possible to predict the moving correlation between Bitcoin and a traditional asset (all assets are considered individually; the goal is not to identify an average traditional asset)? Compare machine learning models with classical economic valuation models. 2) Is it possible to turn machine learning forecasts into an early warning signal for real-world investors?

Four research questions are also examined:

1. Is there a correlation between traditional assets and Bitcoin?
2. Which is better: the DCC-GARCH method or machine learning models?
3. Which is better: basic models, such as AR(1), or machine learning models?
4. Do forecasts based on cryptocurrency data contain practically useful information about stress in traditional markets?

1.3 Contributions of the thesis

In this work, a reproducible Python pipeline was designed from scratch and hosted on GitHub. When run, it loads price data from open exchange sources, constructs logarithmic return series, and calculates rolling correlations for various window lengths. The models used include the naive basic random walk model, AR(1) autoregression, the HAR autoregressive model, regularized linear models such as Ridge and ElasticNet, tree ensembles (Random Forest, GBM, XGBoost), and the DCC-GARCH method with data leakage protection. After the first layer completes its work, the second level performs calculations using the first layer's forecasts to determine whether the market is under stress or not.

The main thing that we found out is that we can predict the further development of the addiction process, but most of this predictability is due to what happened in the past. Adding new information from other sources does not bring significant benefits. The Ridge, AR(1), and HAR models eventually turned out to be approximately equally accurate, since they all rely on the fact that the sliding correlation remains very stable over time. Interestingly, these three completely different models came to the same result — this indicates that the objective function has a fairly simple linear structure, and complicating the models does not really add anything. At the same time, machine learning does bring some benefits, especially when it comes to classifying stressful periods. In this case, regularized linear classifiers work better than ensembles of trees.

This study makes three main contributions. First, we developed a reproducible pipeline in Python that covers the entire analysis process from processing the raw data to generating the final plots. Second, we evaluated ten forecasting models, ranging from simple benchmark specifications to more complex approaches such as XGBoost and DCC-GARCH. This provides a consistent framework for assessing whether the added complexity of machine learning methods leads to improved forecasting performance. Third, we used correlation forecasts to build a simple stress scenario classifier, linking the forecasting results to practical applications in the field of risk management.

1.4 Thesis outline

Section 2 provides an overview of the theory and literature: time-varying dependence, financial contagion, machine learning-based forecasting methods, and the Dibold–Mariano test. Section 3 describes the data, the construction of the target series, feature engineering, model specifications, and the estimation procedure. Section 4 presents the empirical results for both levels. Section 5 interprets and refines these results. Section 6 summarizes the findings.

2 Theoretical Background and Literature Review

2.1 Why intermarket dependency matters

Asset correlations matter a lot in finance. This is because asset correlations change how you handle diversification, hedging, spread and portfolio risk. Standard mean-variance models assume that asset correlations stay the same all the time.. In the real world asset correlations change quite often. Asset correlations usually react harder, to bad news. They also tend to jump up fast when the markets go down. This happens at the moment when diversification is needed the most [1]. Because of this trying to guess how asset correlations will change is a way to manage risk.

These factors become even more significant when one component of the pair is a cryptocurrency [2]. Bitcoin is constantly traded on a market accessible to anyone around the world. It reacts quickly to optimistic or pessimistic sentiment, as well as to a reduction in the money supply. People have their own ideas about what Bitcoin is. Some see it as a hedge against inflation, others as a betting instrument, still others as an indicator of the level of risk people are willing to take, and yet others as something that changes depending on rules and laws. Because it is influenced by many factors, Bitcoin does not occupy a clear position on the investment spectrum. Depending on the stage of a business's or loan's life cycle, Bitcoin can act as a high-risk stock, an indicator of cash availability, or a tool for gauging public sentiment. It is precisely because of this volatile behavior that the relationship between Bitcoin and traditional investments is an example of a phenomenon that changes over time.

2.2 Cryptocurrencies and conventional assets

Over the past ten years, research at the intersection of the cryptocurrency and traditional markets has expanded rapidly. This research can be divided into several main areas. The first area focuses on Bitcoin itself. It examines how Bitcoin returns are distributed. It also examines Bitcoin's volatility and analyzes event risk. It examines whether Bitcoin is inefficient. The second section asks whether Bitcoin can help diversify a portfolio. It considers whether Bitcoin is a good complement to stocks, gold, oil, or major currencies. The third section is the one most closely related to this thesis. It examines how different markets are interconnected. It explores how problems in one market can spread to others. It focuses on how returns and volatility vary across markets. It also focuses on crisis and stress situations.

A recurring finding in this research is that the relationship is not stable [3, 4]. In markets Bitcoins connection with overall stock markets is weak or not clear in direction; in strong risk-, on or risk-off situations it becomes much stronger [5, 6]. Precious metals move with Bitcoin strongly and less consistently [7, 8] which makes sense because they have a group of investors and have long been seen as safe places to put money. Currencies add another aspect acting as a sign of money availability [4].

These patterns suggest that the relationship should be viewed as changing over time rather than as constant. Using a single correlation estimate for the entire sample would obscure important features of the underlying dynamics. A time-varying approach allows us to identify changes in market regimes and sudden disruptions, and it also shows that different asset pairs and the duration of the period can change at different rates and in

different directions.

2.3 Rolling correlation as a forecasting target

We use the moving-window correlation method because it is intuitive and easy to compute. Given two return series, $r_{1,t}$ and $r_{2,t}$, the correlation at time t within a window of length w is defined as

$$\hat{\rho}_t^{(w)} = \frac{\sum_{s=t-w+1}^{t}(r_{1,s} - \bar{r}_{1,t})(r_{2,s} - \bar{r}_{2,t})}{\sqrt{\sum_{s=t-w+1}^{t}(r_{1,s} - \bar{r}_{1,t})^2} \cdot \sqrt{\sum_{s=t-w+1}^{t}(r_{2,s} - \bar{r}_{2,t})^2}}, \tag{1}$$

where $\bar{r}_{k,t}$ denotes the in-window mean of series k. Traversing this estimator across all t yields a time series $\{\hat{\rho}_t^{(w)}\}$ that characterises the temporal evolution of intermarket co-movement.

As a forecasting tool, moving correlation offers several advantages. It is easy to interpret: positive values indicate that two assets tend to move in the same direction, while negative values indicate that they tend to move in opposite directions and may offer hedging opportunities. Correlation can also be compared across different asset pairs and time intervals, making it possible to systematically analyze multiple assets. Finally, when using moving correlation, the focus is on changes in the relationship between assets, rather than on the level of any single return series.

Its main challenge is strong serial persistence. Since $\hat{\rho}_t^{(w)}$ is calculated using overlapping windows, consecutive estimates share $w - 1$ observations. As a result, the series exhibits a high degree of autocorrelation by construction. This persistence can affect forecasting performance and leave complex models with little to add beyond a simple autoregressive specification. In this thesis, we treat this persistence as an important empirical characteristic of the dependency process and analyze it directly.

To improve the statistical behaviour of the target, the thesis applies the Fisher z-transformation:

$$z_t^{(w)} = \frac{1}{2} \ln\left(\frac{1 + \hat{\rho}_t^{(w)}}{1 - \hat{\rho}_t^{(w)}}\right), \tag{2}$$

which stretches the bounded values $\hat{\rho} \in (-1, 1)$ onto the whole real line and gives them a sampling distribution much closer to Gaussian. It is standard practice for correlation-like quantities and tames the variance that otherwise blows up near ± 1.

The main alternative is a copula-based approach [9], in which the marginal distributions and the structure of the dependence are modeled separately. This allows for the dependence in the tails of the distribution and nonlinear joint behavior, which cannot be captured using Pearson's correlation. This dissertation retains the moving Pearson correlation for four reasons: (i) it is readily understandable to risk managers; (ii) it is computationally efficient enough to be recalculated at each step of the "walk-forward" cycle; (iii) it is consistent with the benchmark DCC-GARCH model, which yields the conditional correlation matrix R_t [10]; and (iv) the Fisher z-transformation already accounts for behavior at the distribution boundaries. The use of Pearson's correlation is a practical limitation, and extending the analysis to methods based on copulas is a natural next step.

2.4 Econometric benchmark: DCC-GARCH

The Dynamic Conditional Correlation GARCH model, introduced by [11], provides the classical econometric benchmark for the present study. The DCC-GARCH framework decomposes the conditional covariance matrix H_t of a vector of returns $\mathbf{r}_t$ according to

$$H_t = D_t R_t D_t, \tag{3}$$

where $D_t = \mathrm{diag}(h_{1,t}^{1/2}, h_{2,t}^{1/2})$ is a diagonal matrix of conditional standard deviations and R_t is the time-varying conditional correlation matrix. Each conditional variance $h_{k,t}$ follows a univariate GARCH(1,1) process,

$$h_{k,t} = \omega_k + \alpha_k r_{k,t-1}^2 + \beta_k h_{k,t-1}, \tag{4}$$

with $\omega_k > 0$, $\alpha_k, \beta_k \geq 0$ and $\alpha_k + \beta_k < 1$ for covariance stationarity. The standardised residuals $\tilde{\varepsilon}_t = D_t^{-1} r_t$, which have unit conditional variance by construction, are then used to update the pseudo-correlation matrix Q_t via

$$Q_t = (1 - a - b)\bar{Q} + a\,\tilde{\varepsilon}_{t-1}\tilde{\varepsilon}_{t-1}^{\top} + b\,Q_{t-1}, \tag{5}$$

where $\bar{Q}$ is the unconditional covariance of standardised residuals, and the scalar parameters $a \geq 0$, $b \geq 0$, $a + b < 1$ govern the speed of mean reversion and the persistence of conditional correlation. The conditional correlation matrix is recovered as

$$R_t = \mathrm{diag}(Q_t)^{-1/2}\, Q_t\, \mathrm{diag}(Q_t)^{-1/2}. \tag{6}$$

DCC-GARCH serves as a benchmark in this situation because it shows how everything changes over time as volatility fluctuates. This type of volatility is characteristic of both cryptocurrencies and the stock market. The problem with DCC-GARCH lies in how it is configured. A fixed approach cannot be easily adapted to handle patterns or sudden changes. Another issue, which is harder to spot, is that information from the future gets mixed in. This happens when the model is first trained on all the data and then used to make a forecast for a specific time period. These forecasts then incorporate information that should not be available. To prevent this, the thesis rebuilds the DCC-GARCH model using data from each round of testing. This is done in the same way as for machine learning models.

2.5 Machine-learning approaches to forecasting

Machine-learning methods are included here for their ability to capture nonlinearities, threshold effects, and interactions that are difficult to specify in a parametric model [12]. They are not assumed to outperform in every case; the thesis evaluates them against both the DCC-GARCH benchmark and simple persistence baselines, with two aims: to identify where their strengths lie, and to determine the conditions under which they add predictive value.

2.5.1 The HAR model and persistence-based forecasting

The Heterogeneous Autoregressive (HAR) model, introduced by [13], provides a natural benchmark for forecasting persistent financial time series. The HAR model approximates

long-memory dynamics by summing three autoregressive components operating at daily, weekly, and monthly horizons:

$$z_t = \phi_0 + \phi_d z_{t-1} + \phi_w \bar{z}_{t-5:t-1} + \phi_m \bar{z}_{t-22:t-1} + \varepsilon_t, \tag{7}$$

where $\bar{z}_{t-k:t-1}$ is the arithmetic mean of z over the preceding k periods. The model is particularly well suited to rolling-correlation targets, which already carry a moving-average smoothing from their overlapping windows, the structure HAR is designed to exploit [14]. In this implementation, HAR is a restricted OLS on three regressors, the latest value (daily), the 5-day mean (weekly), and the 22-day mean (monthly), evaluated under the same walk-forward protocol as the other models.

2.5.2 Regularised linear models

Regularised linear models, including Ridge regression and Elastic Net, represent strong baselines in structured tabular forecasting settings. Ridge regression imposes an ℓ_2 penalty on the coefficient vector, shrinking estimates toward zero to reduce estimation variance without introducing sparsity. Elastic Net combines ℓ_1 and ℓ_2 regularisation:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \left[(1-\alpha) \|\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1 \right] \right\}, \tag{8}$$

where $\lambda > 0$ sets the overall strength of the penalty and $\alpha \in [0, 1]$ splits it between the two terms [15]. The ℓ_1 component drives some coefficients to zero, performing implicit variable selection, which is useful when many lagged predictors are weak or redundant. This is well suited to the feature set used here, which contains many overlapping transformations of correlated return and volatility series.

Even though these models are linear these models perform well on persistence-driven problems. When the signal-to-noise ratio is low and the informative feature is a lag of the target the bias from the linearity assumption is usually less than the drop, in variance that regularisation provides.

2.5.3 Tree-based ensemble models

Random Forests construct an ensemble of decision trees, each trained on a bootstrapped subsample and a random subset of features, and aggregate their predictions to reduce variance and improve generalisation [16]. Gradient Boosting Machines instead build trees sequentially, with each new tree fitted to the pseudo-residuals of the current ensemble [17]. XGBoost extends this boosting framework with second-order gradient approximations, ℓ_1 and ℓ_2 regularisation terms on tree structure, and computational optimisations that facilitate large-scale application [18].

Tree-based ensembles are a natural choice, since the relationship between cryptocurrency characteristics and cross-market dependence may be nonlinear and involve interactions between variables. For example, the information contained in Bitcoin's realized volatility regarding future dependence on stocks may depend on how high the current correlation is. Tree-based methods can automatically capture such conditional relationships without requiring interactions to be specified in advance.

However, this does not guarantee that tree-based methods will perform well when forecasting one step ahead for a target variable with a high degree of stability. If the target

variable is determined primarily by its most recent value, the additional flexibility of these models may provide little additional predictive information while increasing the variance of the estimates. This trade-off is one of the main reasons why all models are evaluated using the same step-by-step procedure rather than on the basis of a single summary statistic.

2.6 Walk-forward evaluation and information leakage

Temporal ordering is fundamental in financial forecasting and constrains how the evaluation must be designed. A random train-test split is inappropriate, because it disrupts the causal ordering and allows a model trained on period $t + k$ to "predict" period t. A valid design is one in which every forecast uses only information available at or before its own origin.

Thus, the dissertation employs a step-by-step procedure: the model is trained based on an expanding history, generates a one-step-ahead forecast, then the training sample is extended by one day, and the process is repeated. To reduce computational costs, the models are updated at regular intervals rather than at every step. This is a practical compromise that also reflects how a forecasting system might operate in practice. In any case, information about the future cannot be included in the forecast.

The evaluation protocol is as vital as the selection of a model. A complex model that hides a data leak may look better, than a model that has been evaluated properly and that would break the comparison. Because the goal is to reach conclusions it is mandatory that the evaluation protocol has no data leak.

2.7 Model comparison and the Diebold–Mariano test

Ranking models by average out-of-sample error only goes so far: it says nothing about whether a gap is real or just sampling noise. The Diebold–Mariano test closes that gap, formally testing the null that two forecasts are equally accurate [19].

Let $e_{1,t}$ and $e_{2,t}$ denote the forecast errors of models 1 and 2 at time t, and let $d_t = g(e_{1,t}) - g(e_{2,t})$ denote the loss differential under a loss function $g(\cdot)$ (typically squared or absolute error). The DM test statistic is

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\hat{V}(d)\,/\,T}} \xrightarrow{d} \mathcal{N}(0,1), \tag{9}$$

where $\bar{d} = T^{-1}\sum_{t=1}^{T} d_t$ is the mean loss differential, $\hat{V}(d)$ is a heteroskedasticity- and autocorrelation-consistent (HAC) estimator of the long-run variance of the loss-differential series, and T is the number of evaluation periods; the ratio $\hat{V}(d)/T$ is therefore the estimated variance of $\bar{d}$. Under the null of equal predictive accuracy, the statistic is asymptotically standard normal.

In this case, the DM test serves two purposes. First, it assesses whether the most effective machine learning configuration truly outperforms the DCC-GARCH test in terms of protection against leaks. Second, it assesses whether any improvement over a simple autoregressive model or the baseline level of naive robustness remains significant after accounting for sampling uncertainty. The second comparison is more important: if the

model cannot significantly outperform AR(1), then most of the predictability depends on the asset's own stability, while cross-asset-related functions provide little predictive value.

2.8 From dependency forecasting to investor signaling

Accurate forecasting of rolling correlation would in itself constitute a complete academic exercise. Because the thesis also has a practical motivation, however, it adds a further step: converting the forecasts into a form on which an investor can act, framed around the risk-management decision itself.

This step is implemented as a second-stage warning level. It does not predict the direction of tomorrow's returns. Instead, it assesses the probability that the following day will enter a "stress mode," defined as a period of heightened risk of a decline in value for the traditional asset in question. This formulation is closer to how risk management works in practice, where the goal is to identify deteriorating conditions early enough to take corrective action. Predicting every minor price change is not the objective.

Separating these two stages makes the development process more transparent. A lower forecast error and actual usefulness for portfolio decision-making are distinct properties, and combining them may lead to an overestimation of the results. The two-tier structure allows the dissertation to present a report on each aspect separately.

2.9 Positioning of the present thesis

The thesis lies at the intersection of financial econometrics, machine learning, and applied risk analysis. It does not claim that cryptocurrency data solve the market-timing problem. The claim is narrower still than it might appear: intermarket dependence is forecastable out of sample, but the results in Chapter 4.1 attribute almost all of that forecastability to the persistence of the dependency series itself, with Bitcoin's price dynamics adding only marginally beyond it. What the crypto features do support, modestly, is an early-warning signal for stress days in conventional markets.

The main contribution of this study is to demonstrate that dynamic intermarket relationships can be forecasted, as well as to provide a comparative analysis of the stability metrics of machine learning models and DCC-GARCH models based on a step-by-step approach. The level of investor signals expands the scope of the work in a certain direction but does not require more than what the facts confirm.

3 Methodology

3.1 Research design

The project consists of two parts. The first part is a regression problem: to predict the value of the (adjusted) moving correlation between Bitcoin and another asset. The second part is a classification problem: to use this forecast, which incorporates certain cryptographic features, to calculate the probability that a traditional asset will have a "day." This distinction highlights the difference between observing something and reacting to it.

The design preserves temporal ordering in target construction, feature alignment, benchmark estimation, and signal generation: each is performed inside the walk-forward loop. Model hyperparameters, however, were selected once using time-series cross-validation on the BTC–S&P 500, $w = 30$ representative experiment. Consequently, the reported result for that same experiment is an exploratory illustration rather than a fully independent confirmatory estimate. The all-pair, all-window rankings remain out-of-sample conditional on these fixed choices, but a future study should tune within each training fold (nested walk-forward validation) before making confirmatory claims.

3.2 Asset universe and data source

The empirical analysis uses daily market data obtained through the yfinance interface [20]. Bitcoin, represented by BTC-USD, serves as the base cryptocurrency throughout the study. The comparison set includes the following assets:

Table 1: Asset universe used in the thesis.

Ticker	Interpretation
`BTC-USD`	Bitcoin, base cryptocurrency
`ETH-USD`	Ethereum, crypto reference asset
`^GSPC`	S&P 500 index
`^IXIC`	NASDAQ Composite index
`GLD`	Gold ETF
`SLV`	Silver ETF
`UUP`	U.S. Dollar Index ETF proxy

Each asset is in the set for a reason. The two equity indices stand in for broad U.S. risk appetite. The gold and silver ETFs bring in precious-metal exposure, with its mix of inflation, safe-haven, and commodity narratives. The dollar-index proxy captures global dollar strength and defensive positioning. Ethereum is there so the study can contrast crypto-to-crypto dependence with crypto-to-conventional dependence.

The sample runs from 9 November 2017 to 16 May 2026, giving 3,053 daily observations per asset.[1] The start date is set by Ethereum: Yahoo Finance has no ETH-USD prices before then, so earlier dates drop out of the aligned panel. What remains is balanced: all

[1]The pipeline downloads data to the most recent available trading day; the figures reported here correspond to the final run of the pipeline at the time of writing.

seven tickers share one date index, with no gaps left after alignment and forward-filling of non-trading days up to two consecutive calendar days.

Normalised price paths, thirty-day rolling-volatility profiles, and the full-sample return correlation matrix for the asset universe are collected in Appendix A (Figures 6, 7, and 8). The much higher, more episodic volatility of the crypto assets and the low static Bitcoin–conventional correlations they display are the reason cross-asset dependency is treated here as a moving quantity instead of a single full-sample number.

3.3 Return construction and preprocessing

Every price series is turned into daily log returns. Log returns are the natural choice here: they add up over time and are the standard currency of both econometric and machine-learning work on financial series. For a price series P_t, the return is

$$r_t = \log P_t - \log P_{t-1}$$

Missing observations are dealt with by aligning on common dates; nothing downstream (no target, no feature) is built until the return matrix is synchronised.

Preprocessing is cache-aware. If the raw prices are already on disk, they are reused, which keeps runs reproducible; if the price structure or the ticker set changes, the processed returns are rebuilt from scratch. That way nothing stale lingers, and nothing is recomputed needlessly.

3.4 Target construction: rolling dependency

For each pair consisting of Bitcoin and another asset, a rolling correlation series is computed over windows of 14, 30, 60, and 90 calendar days. Let $r_t^{(b)}$ denote the return of Bitcoin and $r_t^{(a)}$ the return of the other asset. For a given window length w, the dependency target at time t is

$$\rho_t^{(w)} = \operatorname{corr}\Big(\{r_{t-w+1}^{(b)}, \ldots, r_t^{(b)}\}, \{r_{t-w+1}^{(a)}, \ldots, r_t^{(a)}\}\Big).$$

The resulting series is then transformed using the Fisher transformation

$$z_t^{(w)} = \frac{1}{2} \log\left(\frac{1+\rho_t^{(w)}}{1-\rho_t^{(w)}}\right) = \operatorname{arctanh}\Big(\rho_t^{(w)}\Big),$$

which maps the bounded correlation domain $(-1, 1)$ into an unbounded real-valued space. In the empirical implementation, the one-step-ahead value of the transformed dependency series is used as the main forecasting target. The implementation clips the raw correlation away from the boundary to prevent numerical overflow:

Listing 1: Fisher z-transform and target construction (`pipeline.py`).

```
def fisher_z(r: pd.Series, eps: float = 1e-6) -> pd.Series:
    # Clip away from +/-1 to prevent arctanh overflow
    return np.arctanh(r.clip(-1 + eps, 1 - eps))

```

```python
def build_target(returns, base, other, window, horizon):
    corr = returns[base].rolling(window).corr(returns[other]).dropna()
    # Shift by horizon to get the one-step-ahead target
    target = corr.shift(-horizon).dropna()
    return fisher_z(target)
```

One consequence of all this is that the target is smooth to begin with, and far more persistent than a raw return series. That persistence is one of the central empirical facts of the thesis, and it shows up everywhere in the results.

The rolling dependency series for all six pairs across the four window lengths is shown in Figure 1. Its pronounced time-variation and persistence are the properties the forecasting framework is designed to exploit.

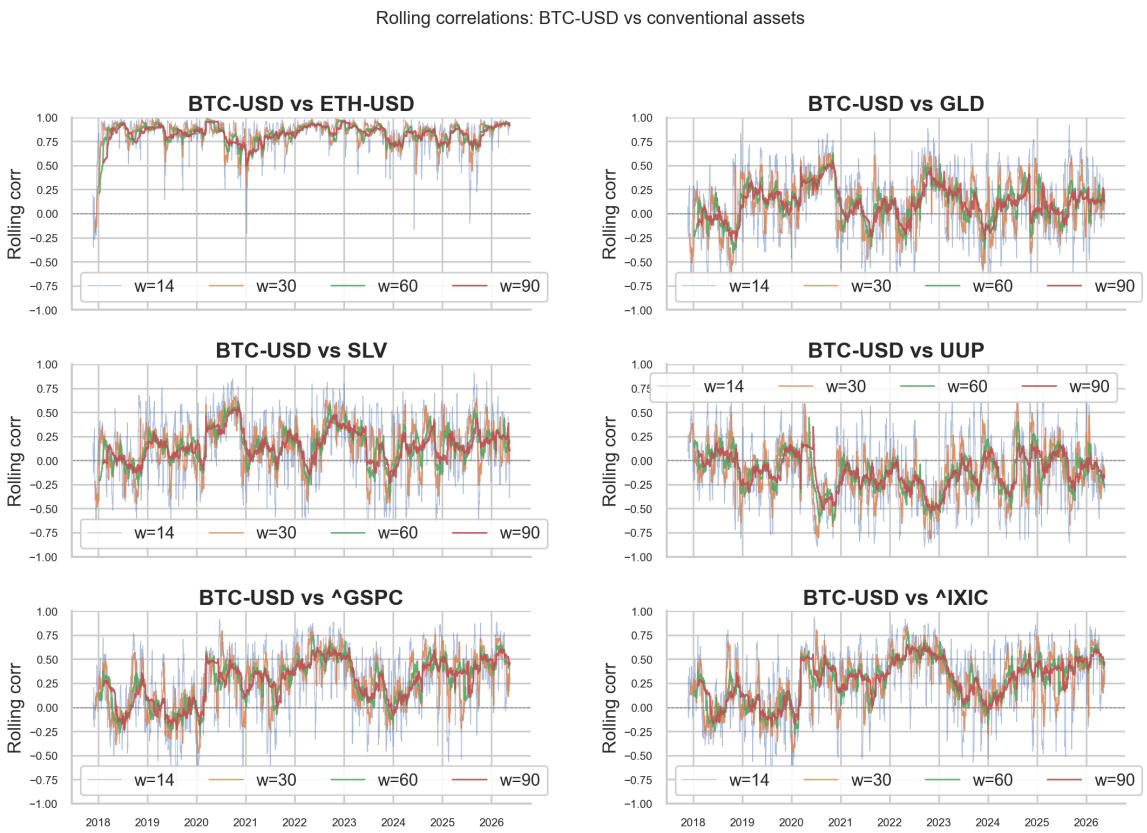


Figure 1: Rolling dependency between Bitcoin and each of the five conventional assets plus Ethereum (six pairs in total) across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series exhibits the strong time-variation and persistence that motivate the forecasting framework.

3.5 Feature engineering

The features are built around quantities a financial reader can interpret: a compact set of variables that could plausibly move future dependency, kept small enough to stay legible.

The features fall into several groups:

Table 2: Main groups of predictors used in the dependency-forecasting layer.

Group	Description
Dependency lags	Recent values of the rolling dependency target
Dependency momentum & regime	Trend (5- and 20-day change) and trailing z-score of the target
Return lags	Lagged returns of Bitcoin and the paired asset
Rolling means	Local average return over short windows
Rolling volatility	Local standard deviation of returns
Spread features	Absolute and signed gaps between crypto and asset behaviour
Correlation regime gap	Short-window minus long-window correlation

The dependency lags carry most of the predictive weight, because the target is persistent by construction. Dependency momentum (dep_mom) and the trailing z-score (dep_zscore) summarise the recent trend and the standardised level of the correlation. Return lags allow the models to respond to recent price direction, while the rolling-mean, volatility, and dispersion features describe the local risk regime. The regime gap (corr_diff) measures how far short-term dependency has moved from its longer-term baseline, which can act as an early indicator of a regime shift. Listing 2 shows how the feature matrix is assembled.

Listing 2: Core feature engineering routine (`pipeline.py`; abbreviated — the full routine additionally builds rolling-mean, dispersion, volatility-ratio, and lagged squared-return features).

```
# Dependency lags (extended: up to quarterly)
for lag in [1, 2, 5, 10, 20, 60]:
    features[f"dep_lag{lag}"] = y_current.shift(lag)

# Dependency momentum (trend of the correlation series)
features["dep_mom_5"]  = y_current.diff(5)
features["dep_mom_20"] = y_current.diff(20)

# Dependency regime: z-score vs trailing 252-day history
features["dep_zscore"] = (y_current - dep_roll_mean) / (dep_roll_std + 1e
    ↪ -8)

# Volatility of both assets, at the pair window and at 60 days
features["vol_base"]     = returns[base].rolling(window).std()
features["vol_other"]    = returns[other].rolling(window).std()
features["vol_base_60"]  = returns[base].rolling(60).std()
features["vol_other_60"] = returns[other].rolling(60).std()

# Return lags for both assets
for lag in [1, 2, 5, 10, 20]:
    features[f"r_base_lag{lag}"]  = returns[base].shift(lag)
    features[f"r_other_lag{lag}"] = returns[other].shift(lag)

```

```
# Correlation regime gap: short-window minus long-window correlation
corr_short = rolling_corr(returns[base], returns[other], max(5, window //
    ↪ 4))
corr_long  = rolling_corr(returns[base], returns[other], window)
features["corr_diff"] = corr_short - corr_long
```

The signal layer takes the best available dependency forecast for the pair and combines it with a handful of crypto-derived predictors into a smaller, classification-oriented feature set. In effect, the second layer sees both the current state of the crypto market and where the dependency process is heading.

3.6 Forecasting models

3.6.1 Baseline and autoregressive models

Three persistence-based models anchor the set. The naive last-value baseline just carries the last observed value forward, with no fitting at all. AR(1) fits a single autoregressive lag by OLS. HAR (Heterogeneous AutoRegressive, after [13]) adds two more horizons: the latest value, its five-day average (weekly), and its twenty-two-day average (monthly), fitted as a restricted OLS on the expanding window at each refit. These baselines are not a formality: because the target is persistent by construction, any more complex model must demonstrate a clear improvement over them to justify its use.

3.6.2 Regularised linear models

The linear block is Ridge and ElasticNet, each fitted after standardisation (regularisation is meaningless if the features sit on different scales). Both are specified in the notation of Eq. (8): λ is the penalty strength and α the ℓ_1/ℓ_2 mixing ratio, which scikit-learn's interface calls alpha and l1_ratio respectively. Ridge (ℓ_2, $\lambda = 0.5$) should do well when many predictors each carry a little signal. ElasticNet ($\lambda = 0.005$, $\alpha = 0.5$) is more aggressive and can zero out redundant lags outright.

3.6.3 Tree ensembles and adaptive ensemble

The non-linear block is Random Forest, HistGradientBoostingRegressor (a LightGBM-style histogram method, roughly ten times faster than the old gradient-boosting code), and XGBoost, which runs on the GPU when it can and quietly falls back to CPU when it cannot. On top of these sits an Ensemble: at each step it forms an inverse-RMSE weighted average of all the non-naive models, with the weights recomputed from the last 60 out-of-sample errors (during the first 60 steps, from whatever errors have accumulated so far).

The complete model set, ten specifications in total, is initialised as follows:

Listing 3: Model definitions used in the walk-forward experiment (`pipeline.py`; simplified — `Naive_Last` and `HAR` are `None` here and are produced directly inside the walk-forward loop rather than through `.fit`).

```
model_specs = {
    "Naive_Last": None,          # persistence baseline (no fit required)
    "AR(1)":        LinearRegression(),
```

```
    "HAR":        None,          # heterogeneous AR -- fitted via OLS on 3 lags
                                 # (dep_lag1, avg5, avg22); see walk-forward loop
    "Ridge":      make_pipeline(StandardScaler(), Ridge(alpha=0.5)),
    "ElasticNet": make_pipeline(
                      StandardScaler(),
                      ElasticNet(alpha=0.005, l1_ratio=0.5, max_iter=5000)
                  ),
    "RF":  RandomForestRegressor(
               n_estimators=200, max_depth=10,
               min_samples_leaf=5, n_jobs=-1
           ),
    "GBM": HistGradientBoostingRegressor(     # LightGBM-style histogram algorithm
               max_iter=300, max_depth=4,
               learning_rate=0.02, min_samples_leaf=5,
               l2_regularization=0.1
           ),
    "XGB_GPU": xgb.XGBRegressor(
               device="cuda", n_estimators=500,
               max_depth=6, learning_rate=0.02,
               subsample=0.8, colsample_bytree=0.8
           ),
}
# "Ensemble": adaptive inverse-RMSE weighted average of the ML models
#             (excludes Naive_Last and the DCC-GARCH benchmark).
# Weights are recomputed at each step from the last 60 out-of-sample errors .
```

3.6.4 DCC-GARCH benchmark

The DCC-GARCH benchmark is implemented in a separate module and is leakage-safe by construction. Fitting DCC once on the full sample and then reading off "out-of-sample" predictions would use future information, so the benchmark is instead re-estimated on the training window alone and rolled forward from there. This is computationally slower, but it places the benchmark on the same out-of-sample footing as the machine-learning models.

3.7 Investor signal layer

The signal layer reformulates the problem as early-warning classification. The label is a stress day for the conventional asset, not the direction of the next-day return. Day $t+1$ is labelled as stress if the asset's return falls below $-0.75\,\sigma_t$, where σ_t is the trailing 20-day standard deviation of returns computed from information available at time t. This threshold produces a positive (stress) class of roughly 15–18% of days, depending on the asset (15.2% for the S&P 500 up to 17.5% for the dollar index), and its sensitivity is examined over the range $[0.5,\ 2.0]\,\sigma$ in the robustness analysis. The threshold is chosen to capture moves that are worse than a typical down day without being limited to extreme tail events, which matches the downside-protection use case of the signal.

Three classifiers compete here: logistic regression, a random-forest classifier, and a gradient-boosted one. Each emits a probability, and a day is flagged as elevated-risk when that probability is at least 0.55. This fixed operating threshold is used for the reported classification metrics and signal plots. The output is therefore a probability-guided warning, not an opaque buy/sell command.

3.8 Walk-forward estimation framework

Every experiment uses the same expanding-window walk-forward. Start with the first T_0 observations; at each origin $t \geq T_0$, fit on everything up to t and predict $t + 1$; continue to the end of the sample. Refitting happens periodically instead of at every step, which cuts the compute cost without ever breaking causal order.

Listing 4: Expanding-window walk-forward loop (`pipeline.py`).

```
for t in range(min_train, n_obs):
    train_idx = np.arange(0, t)

    # Refit all models on the expanding training window periodically
    if (t - last_refit) >= refit_every:
        last_refit = t
        for name, model in model_specs.items():
            if model is not None:
                model.fit(X_values[train_idx], y_values[train_idx])
                fitted[name] = model

    # One-step-ahead prediction - uses only information up to t
    for name in fitted:
        preds[name][t] = fitted[name].predict(X_values[[t]])[0]
```

This design has three benefits: it reproduces realistic forecasting conditions, it keeps the benchmark comparison fair, and it produces a complete time series of out-of-sample errors. The last point is important, since those errors feed not only the average metrics but also the rolling diagnostics and the Diebold–Mariano tests.

3.9 Evaluation metrics

The forecasting layer is evaluated with three metrics: mean absolute error (MAE), root mean squared error (RMSE), and the out-of-sample R^2. Rankings are based on RMSE, which penalises large errors more heavily and is well suited to a smooth, continuous target.

For the signal layer, four numbers matter: Balanced Accuracy, the F1 score on the downside class, the area under the ROC curve (AUC), and the exit rate. Balanced Accuracy is emphasised because stress days are rare and the classes are imbalanced. The F1 score on the downside class keeps the focus on detecting the adverse days. The exit rate (how often the signal would move the investor out of the market) links statistical quality to how the signal would actually be used.

3.10 Model comparison methodology

When models are close, average metrics can mislead. So the thesis runs the Diebold–Mariano test on the loss series of selected pairs. The headline comparison is the best non-benchmark machine-learning model against DCC-GARCH; the secondary one pits that same model against the naive persistence baseline.

The setup is transparent by design. The thesis does not hide the possibility that a simple baseline may be the best overall model. Instead, it uses the DM framework to distinguish between three different levels of claims:

1. whether dependency is forecastable at all,
2. whether machine learning improves upon a classical econometric benchmark,
3. whether an observed RMSE difference is statistically distinguishable under the stated loss-differential assumptions.

A caveat applies to comparisons involving nested models. When the simpler model is nested inside the richer one (as the naive baseline and AR(1) are inside Ridge or the tree ensembles), the standard Diebold–Mariano statistic is not strictly valid, because its asymptotic normal distribution assumes non-nested forecasts. Tests built for nested models, such as Clark–West or Giacomini–White [21, 22], would be more rigorous, and the small-sample HLN correction reported in Appendix E mitigates over-rejection but not nesting. The comparisons against persistence baselines are therefore indicative only. In addition, the reported DM statistics use a Newey–West HAC estimator with zero lags. Although forecasts are one step ahead, the rolling-correlation target uses overlapping windows, so loss differentials can be autocorrelated and zero-lag standard errors can be too small. The reported DM values and p-values are consequently descriptive diagnostics, not formal significance evidence; a confirmatory analysis should use a HAC lag selected for the window-induced dependence and report sensitivity to that choice. The large RMSE gap to DCC-GARCH remains a descriptive finding, but its formal statistical certainty is not claimed here.

The thesis organises its empirical conclusions around exactly these three levels.

3.11 Implementation and reproducibility

The whole workflow is Python, organised as a modular codebase. A main pipeline ties the steps together (read the config, compute returns, build targets and features, fit models, write prediction files, save figures, export LaTeX tables), while separate modules handle the DCC walk-forward benchmark, notebook support, and the signal layer. This organisation is what makes the thesis reproducible end to end.

Everything lands in a structured output tree: metrics, DM tests, and signal results as CSV; thesis-ready tables as LaTeX; diagnostics as figures. The point is to keep manual post-processing to a minimum and let the analysis feed the writing directly.

3.12 Methodological expectations

Before the results, it is worth saying what success would even look like. It does not mean finding a model that predicts every conventional market from Bitcoin; no serious

forecasting study should promise that. A fairer bar has three parts: the dependency target should show out-of-sample forecastability, the benchmark comparison should establish whether machine learning improves on DCC-GARCH, and the signal layer should produce stress warnings that beat chance and can be acted on. These thresholds are modest on purpose.

Three further principles guide how the results are read. First, no single metric is treated as the verdict: the regression layer always reports RMSE, MAE, and R^2 together, and the classification layer reports Balanced Accuracy, F1, AUC, and exit rate together. Second, paired DM statistics complement average loss differences but are interpreted descriptively because of the overlapping target. Third, the analysis refuses to cherry-pick a window after the fact; all four (14, 30, 60, 90 days) are reported side by side, so no finding can rest on a lucky choice of specification.

Behind all three is the same commitment to methodological transparency. In-sample performance can appear strong yet deteriorate substantially under proper out-of-sample testing, and financial machine learning is unusually exposed to this failure, since repeated evaluation on the same historical sample inflates apparent performance long before any single result looks suspicious [23]. The walk-forward design, the leakage-safe benchmark, and the all-windows evaluation together guard against the kind of inflated claims to which this literature is prone.

4 Empirical Results

4.1 Overview of the empirical output

The pipeline provides a comprehensive and internally consistent set of factual data: out-of-sample metrics, model rankings, Diebold–Mariano diagnostics, classification statistics at the signal level, the forecast series themselves, and diagnostic metrics. This breadth is intentional. The empirical results are not based on a single table. Instead, the main conclusions are supported from several perspectives: average forecast accuracy, paired-loss diagnostics, error behavior over time, and the effectiveness of stress scenario warnings.

Two conclusions follow from what is stated below. First, the process of dependence can be predicted based on a sample in accordance with a strict step-by-step plan. Second, almost all of this predictability stems from the object's own stability, while functions related to various assets provide little additional predictive information. Taken together, these two conclusions describe the nature of the forecasting problem and its main limitations.

4.2 Average dependency-forecasting performance

Table 3 gives each model's average out-of-sample performance across all dependency experiments, ranked by RMSE, the metric that reacts most to the large misses that matter on a smooth, bounded target.

Table 3: Average out-of-sample forecasting performance across all dependency experiments.

Model	Average RMSE	95% CI for RMSE	Average MAE	Average R^2
Ridge	0.0656	[0.0611, 0.0701]	0.0419	0.9432
AR(1)	0.0659	[0.0613, 0.0706]	0.0403	0.9424
HAR	0.0659	[0.0613, 0.0706]	0.0405	0.9424
Naive (last value)	0.0666	[0.0619, 0.0714]	0.0395	0.9412
ElasticNet	0.0669	[0.0625, 0.0713]	0.0434	0.9411
Ensemble	0.0694	[0.0652, 0.0738]	0.0462	0.9365
GBM	0.0886	[0.0842, 0.0932]	0.0625	0.8983
Random Forest	0.0905	[0.0860, 0.0952]	0.0636	0.8936
XGBoost	0.1241	[0.1184, 0.1297]	0.0864	0.7923
DCC-GARCH	0.2136	[0.2067, 0.2203]	0.1683	0.3715

Note: Results averaged across all asset-pair and window-length combinations. RMSE and R^2 computed on out-of-sample walk-forward predictions. The interval column averages the per-experiment bootstrap bounds over the same 24 experiments: for each experiment the $(y_t, \hat{y}_t)$ pairs are resampled with replacement 500 times and the 2.5th and 97.5th percentiles of the resulting RMSE distribution are taken (`07_Robustness_Checks.ipynb`, Appendix D). The column describes the typical width of the interval around each model's error, not a pooled confidence interval for the average itself; and because the resampling is i.i.d. over pairs while the forecast errors are serially correlated, the intervals are, if anything, too narrow. HAR = Heterogeneous AutoRegressive model; Ensemble = adaptive inverse-RMSE-weighted ensemble. Best-performing model by average RMSE is Ridge (regularised linear regression with StandardScaler). All metrics reflect the full sample through 16 May 2026 (3,053 observations).

The top three (Ridge, AR(1), and HAR) are nearly indistinguishable, with average RMSE

of 0.0656, 0.0659, and 0.0659 and R^2 of 0.9424 and above. The 0.0003 spread between them is economically meaningless, and the naive last-value baseline sits only just outside at RMSE 0.0666, which already indicates that serial persistence accounts for most of the forecastable variation.

The bootstrap intervals put the spread in perspective. Each of the top four models carries an interval roughly 0.009 wide, against differences between them of 0.0003 to 0.0010, while nothing in the leading cluster comes within 0.01 of GBM or Random Forest and DCC-GARCH sits further away than any of these intervals is wide, by more than an order of magnitude. Overlapping marginal intervals are only suggestive, however, since the models are evaluated on the same observations and their errors are strongly correlated; the question of whether one model genuinely beats another has to be settled on the paired loss differentials.

The paired Diebold–Mariano diagnostics do not support a stable ordering at the top. Ridge, the leader on average RMSE, has a lower loss than the naive last-value baseline in 6 of the 24 experiments, a higher loss in 9, and a similar loss in the remaining 9 under the reported zero-lag DM calculation. HAR shows lower loss in 7, higher loss in 1, and similar loss in 16. Because the rolling targets overlap, these DM classifications are descriptive rather than formal significance tests; their direction nevertheless reinforces the conclusion that the ordering changes with asset pair and window length. The picture below the top tier is different: GBM, Random Forest, and XGBoost have higher RMSE than the naive baseline in all 24 experiments. What the ranking supports is a division into tiers, not an inferential ordering inside the top tier. One nuance should be stated explicitly: the ranking here is by RMSE, on which Ridge leads. On mean absolute error the ordering differs, with the naive last-value baseline formally best (MAE 0.0395), ahead of AR(1) (0.0403), HAR (0.0405), and Ridge (0.0419). RMSE penalises large errors more heavily, so Ridge effectively trades a few more small errors for fewer large ones. ElasticNet and the adaptive Ensemble form a second tier above $R^2 = 0.93$. The tree methods (GBM, Random Forest, XGBoost) trail well behind, with RMSE between roughly 0.09 and 0.12 and R^2 below 0.90. DCC-GARCH finishes last by a wide margin ($R^2 = 0.3715$).

The composition of the top cluster deserves a closer look. Ridge, AR(1), and HAR reach almost the same accuracy through different mechanisms: AR(1) uses a single autoregressive coefficient, HAR combines three time scales (daily, weekly, and monthly), and Ridge fits the full regularised feature set anchored by the most recent lag. They converge because each is ultimately exploiting the same property, the strong persistence of the target, and the additional structure in HAR and Ridge does not capture anything that a single lag has not already accounted for. The convergence says more about the target than about the models: the series is so strongly autoregressive that there is little else left to capture.

4.3 Representative forecast example

Figure 2 shows the out-of-sample forecast panel for Bitcoin–S&P 500 at the 30-day window. It is the natural representative case: of all the pairs, this one ties the crypto market to the broadest gauge of U.S. equity risk.

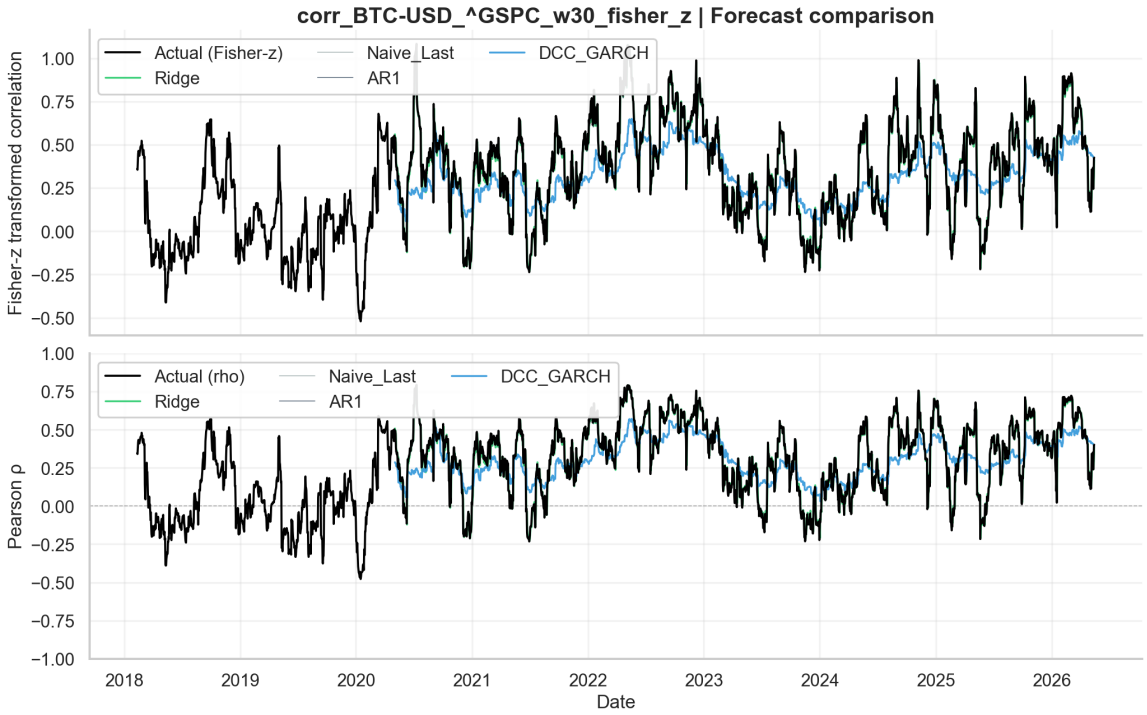


Figure 2: Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information.

The figure highlights three characteristics of the problem. First, the target moves smoothly because it is constructed using overlapping windows. Second, the best models replicate the overall trajectory with a small systematic deviation. The most challenging aspect is capturing inflection points: when the relationship changes abruptly, all models tend to respond inadequately, albeit to varying degrees. During these periods, DCC-GARCH performs the worst, while machine learning gains a relative advantage. However, it still does not outperform simple persistence in terms of overall forecasting accuracy.

4.4 Forecasting performance by asset class

The averages hide some useful cross-sectional variation. Bitcoin–Ethereum is the easiest pair to forecast: the two share liquidity and sentiment drivers, so their correlation is especially smooth and autocorrelated. It posts the highest out-of-sample R^2 of any pair, and longer windows push it higher still.

The precious-metal pairs, Bitcoin–Gold and Bitcoin–Silver, sit in the middle. Their rolling correlation is regime-dependent, drifting between near-zero and weakly positive or negative. Even so, the target stays forecastable, and persistence-based models keep the lead.

The stock index pairs—Bitcoin-S&P 500 and Bitcoin-NASDAQ—are the most relevant in practice. They link cryptocurrency price movements to broader market risk appetite and, on average, are the most predictable among non-crypto pairs. During periods of calm, the relationship is relatively smooth and exhibits strong autocorrelation, which also favors forecasts based on stationarity. However, these pairs are also highly sensitive to changes

in market conditions. Their co-movement may intensify during periods of heightened risk and can change abruptly during market turmoil. These sharp changes lead to random spikes in forecast errors that are not fully accounted for by any model, making them the primary short-term challenge for forecasting stock pairs.

The Bitcoin-Dollar Index pair exhibits the weakest and most specific correlation, as well as the lowest average predictability. Nevertheless, it has been included to test whether a set of cryptographic functions can capture the macroeconomic trend channel at all.

To check that the lower errors on equity pairs are not just luck, the pooled out-of-sample losses of the equity pairs (S&P 500, NASDAQ) are tested against those of the non-equity pairs (gold, silver, dollar index) with a two-sample Diebold–Mariano test [19]. The two groups have different targets, and therefore loss series on different scales, so this pooled comparison is descriptive; it nonetheless indicates that the equity-pair advantage is systematic.

4.5 The persistence dominance result

That simple persistence models lead the average ranking was to be expected.

This result follows directly from the construction of the target. A rolling Pearson correlation over w calendar days shares $w - 1$ observations with the previous window, which smooths the series in a way similar to a moving average. After applying the Fisher transform, the resulting series is strongly autocorrelated, and the first lag already explains most of the predictable variation. As a result, any model that relies heavily on the most recent value can achieve a high out-of-sample R^2 almost by construction.

The Random Forest feature importances point to the same conclusion. The lagged dependency term dominates the ranking in every experiment, while the return lags, rolling volatilities, and the regime gap contribute very little. The problem is forecastable mainly because the target is highly persistent; the features carry no strong cross-asset signal beyond what recent dependency already shows.

4.6 Feature-group ablation

Feature importances describe how a fitted model distributes weight; they do not say what the model would have achieved without a given block of inputs. To measure that directly, the same models are retrained on a nested sequence of feature sets, from a single dependency lag up to the full matrix, under the identical walk-forward protocol, split points, and seeds. Only the columns of the design matrix change between runs. The five sets follow the order in which a practitioner would naturally build the model: the lagged target first, then everything derivable from the history of the dependency series, then the base asset's own returns, then the paired asset's, and finally the cross-asset interaction terms. Table 4 reports the result for the representative Bitcoin–S&P 500 experiment at $w = 30$. The experiment is reproduced end to end by `run_ablation.py` and, in annotated form, by `09_Feature_Ablation.ipynb` (Appendix D).

Table 4: Feature-group ablation, Bitcoin–S&P 500 at $w = 30$. Each row adds a block of features to the row above and keeps everything before it; # is the resulting number of columns in the design matrix.

Feature set	#	Ridge		Random Forest		XGBoost	
		RMSE	R^2	RMSE	R^2	RMSE	R^2
G1 dependency lag only	1	0.0899	0.8849	0.0986	0.8613	0.0980	0.8631
G2 + dependency history	11	0.0649	0.9400	0.0884	0.8886	0.1030	0.8488
G3 + Bitcoin returns	20	0.0649	0.9399	0.0891	0.8868	0.1058	0.8406
G4 + related-asset returns	29	0.0650	0.9397	0.0902	0.8841	0.1078	0.8345
G5 full model	32	0.0651	0.9397	0.0910	0.8821	0.1134	0.8166

*Note: G2 adds the remaining dependency lags, the 5- and 20-day dependency momentum, the 252-day regime z-score, and the multi-scale correlation terms. G3 adds Bitcoin's return lags, rolling volatilities, mean return, and squared return; G4 adds the same block for the S&P 500; G5 adds the volatility ratio and the two absolute-spread terms, which are the only features that combine both assets. Walk-forward settings are those of the main experiment (*min_train = 800*, refit every 20 steps, seed 42).*

Three things follow from the table, and the third is the one worth pausing on.

The first is that the single dependency lag already delivers $R^2 = 0.885$ for Ridge. One column, no cross-asset information of any kind, and most of the forecastable variation is already accounted for.

The second conclusion is that the only functional block that leads to a significant improvement is the model's own history of dependencies. The RMSE drops from 0.0899 to 0.0649 when additional lags, impulse conditions, and the z-score of the mode are added, after which the improvement stops. If we sum the returns on Bitcoin, the RMSE is 0.0649; if we sum the returns on the S&P 500, it is 0.0650; and if we add the conditions of interaction between assets, it is 0.0651. Starting with G2, approximately two-thirds of the design matrix does not contribute significant improvements. This is the result of constancy in its most obvious form: functions related to various assets provide little, if any, additional predictive information for the linear model.

The third conclusion is that, for tree-based ensembles, additional features are not only unhelpful but actually harmful, with performance deteriorating as new features are added. XGBoost performs best with a one-lag specification (RMSE 0.0980), and its RMSE increases with each subsequent step, rising by 16% with the full feature set (0.1134). Random Forest improves after the dependency history is added, but its performance then also deteriorates as additional feature blocks are included. Each additional feature block expands the range of splits that the ensemble can fit. Since the added variables contain little information about the target, this additional flexibility is mainly used to fit noise within the training window. This pattern also explains a result that might otherwise seem unusual in Table (tab): tree-based methods lag behind linear models not because they are poorly suited to the target, but because they are given predictors that only a strongly regularised estimator can safely ignore.

4.7 Comparison with DCC-GARCH

Even though persistence baselines win the unconditional ranking, machine-learning models have lower RMSE than DCC-GARCH by a large descriptive margin. Figure 3 shows the Diebold–Mariano diagnostic between the best machine-learning model and the DCC benchmark across every experiment.

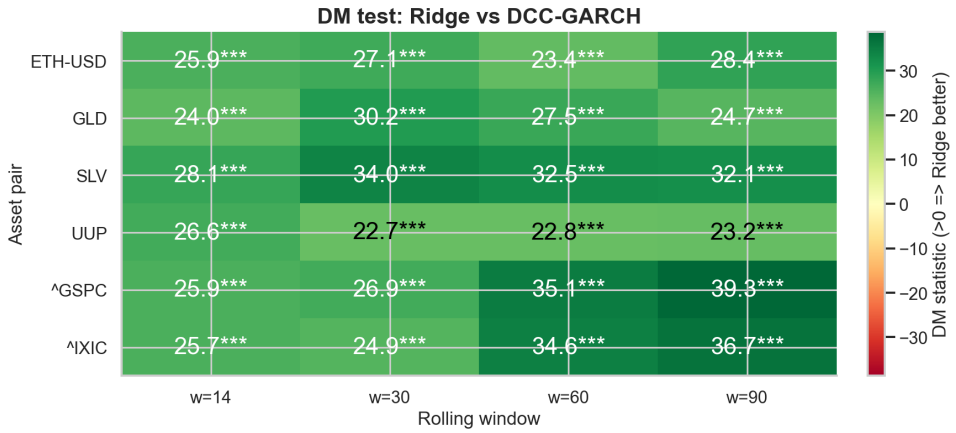


Figure 3: Diebold–Mariano diagnostic comparing Ridge (best overall RMSE across all models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate a larger positive loss-differential statistic; they do not establish formal significance because the reported standard errors use zero HAC lags.

The DM statistics are positive in every reported experiment, consistent with the lower RMSE of the best machine-learning model relative to DCC-GARCH. They are descriptive rather than confirmatory because the reported zero-lag HAC standard errors do not model autocorrelation introduced by the overlapping rolling target. The pattern has a structural explanation: DCC-GARCH is designed to model the conditional covariance of the return series, whereas the rolling Pearson correlation is a smoothed by-product of this covariance and is not targeted directly by DCC-GARCH. In contrast, the machine-learning models are trained directly on the Fisher-transformed correlation target. Thus, they are trained on the same quantity on which they are evaluated, which gives them an advantage.

This comparison involves a measurement mismatch that should be acknowledged. The rolling Pearson target is smoothed and backward-looking, with persistence created by the overlapping windows. DCC-GARCH models the instantaneous conditional covariance as a parametric and forward-looking object, so mapping its output into the rolling Pearson space introduces a systematic lag. This lag follows from the different definitions of the two objects [10, 14]. The comparison remains valid because both are legitimate measures of dependence, but the size of the gap should be interpreted with caution. It should not be taken as a general verdict against DCC-GARCH.

Throughout, the thesis distinguishes two claims: benchmark superiority, meaning improvement over DCC-GARCH, and unconditional superiority, meaning improvement over AR(1). Machine learning achieves the first consistently and the second not at all. Together the two results mark out what machine learning contributes in this setting.

4.8 Error-profile diagnostics

Averages alone do not show when and how a model fails; that requires examining the error over time and across its distribution. Figure 4 plots the rolling RMSE for the representative Bitcoin–S&P 500 experiment.

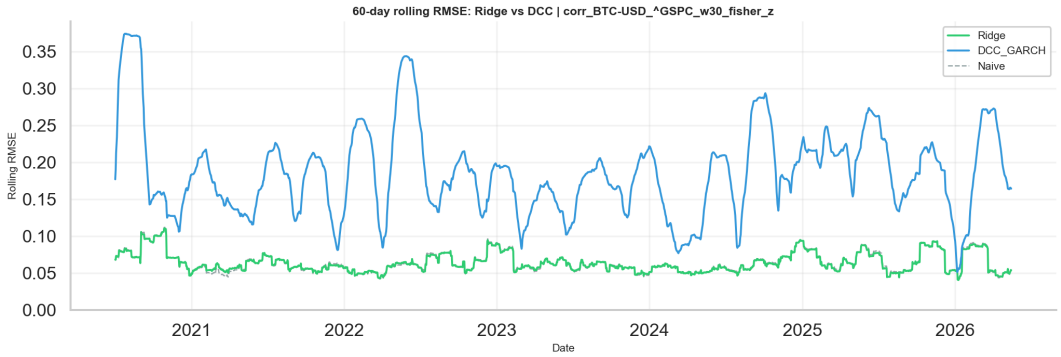


Figure 4: Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment at the 30-day correlation window ($w = 30$); the error is smoothed over a trailing 60-day window. The machine-learning model shown is Ridge, which has the lowest average RMSE overall (Table 3), compared against the DCC-GARCH benchmark. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process.

The quality of forecasts is far from constant over time. When the underlying relationship changes gradually, all models perform well, since it is relatively easy to capture a stable pattern. When conditions change abruptly, errors increase in all models, but to varying degrees and over different time periods. Machine learning models generally recover somewhat faster, which is consistent with their greater flexibility to relearn the relationship between features and the target after structural changes.

The error distribution confirms that the largest errors arise from episodic underreaction at sharp turning points; there is no systematic bias. The models track the slow, persistent component of the dependency reliably and perform worst on the fast, discontinuous component. This asymmetry is directly relevant to the signal layer, whose purpose is to detect the onset of stress in time.

To make this regime-sensitivity concrete, Appendix F summarises eleven landmark events from 2020 to 2026 on a master timeline and a correlation regime map, and provides four-panel diagnostics for four representative episodes: the COVID crash of March 2020, the FTX collapse of November 2022, the SVB banking crisis of March 2023, and the tariff shock of April 2025. Each diagnostic shows how the rolling Bitcoin correlation moved around the event relative to its pre-event baseline. The case studies are consistent with the main finding: the largest errors in Figure 4 coincide with the sharpest events in the record.

4.9 Investor signal layer: average results

The second layer turns the dependency forecast into a stress-warning classifier. Table 5 reports each classifier family's average performance across all asset pairs and windows.

Table 5: Average investor-signal performance by classifier family, aggregated across all experiments.

Signal model	Balanced Accuracy	F1_down	AUC
Logistic regression	0.511	0.214	0.513
GBM (classifier)	0.508	0.158	0.517
Random Forest (classifier)	0.502	0.042	0.519

Note: Results averaged across all asset pairs and window lengths. Balanced Accuracy corrects for class imbalance. $F1_{down}$ measures detection quality for the stress class only.

Classification is a substantially harder problem than the regression layer. Stress days are noisy, rare, and asymmetrically costly to miss. The logistic classifier achieves the highest F1 on the stress class (0.214 on average), which indicates that it remains sensitive to genuine stress without collapsing into an inactive, low-exit-rate state. GBM reaches a similar balanced accuracy (0.508) but a lower F1 (0.158), because it concentrates its probability mass and triggers less often. Random Forest performs poorly, with an F1 of 0.042. Its predicted probabilities concentrate in a narrow band, so it rarely crosses the decision threshold and provides little practical value despite a comparatively high AUC.

This degenerate behaviour merits closer examination. Two factors combine to produce it: the severe imbalance between stress and non-stress days, and the absence of probability calibration. Ensemble methods are well known to produce poorly calibrated probabilities unless they are post-processed with Platt scaling [24] or isotonic regression [25]. GBM, which concentrates its probability mass less severely, retains a modest but non-trivial F1 (0.158). Calibration should therefore precede threshold-based signal extraction. It is not applied anywhere in the results reported here, and integrating it into the pipeline is left for future work; the Random Forest figures in particular should be read as the performance of an uncalibrated classifier rather than as evidence that the method is unsuited to the task.

This result is not surprising. In a setting with a weak signal relative to noise and correlated predictors, a regularised linear boundary can generalise better out of sample than a high-capacity ensemble, whose greater flexibility can increase variance. This is especially relevant when the data are noisy and the predictors are highly correlated.

4.10 Best signal configurations

The highest logistic-regression balanced accuracy occurs for Bitcoin versus the S&P 500 at the 30-day window (0.5308; downside-class F1 = 0.2430). This is only 0.0308 above the uninformative benchmark of 0.50 and is the maximum among twenty asset-window configurations. Since no hypothesis was prespecified and no multiple-comparison correction was applied, it is a descriptive maximum rather than evidence of a reliable advantage. The signal results are therefore reported as exploratory: they motivate further validation on an untouched period, but do not establish a usable performance edge at this configuration.

Figure 5 presents the signal output for this configuration, illustrating the temporal distribution of predicted stress flags relative to realised stress events.

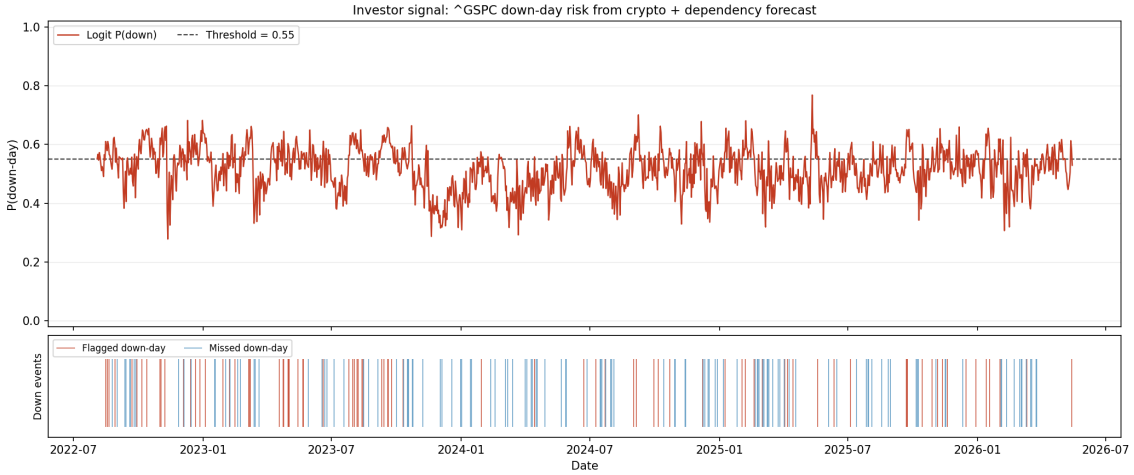


Figure 5: Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events.

This signal is intended to provide probability-based warnings that identify periods of elevated risk and support risk-mitigation decisions. It is not a rule for directional trading. For the NASDAQ, the signal is noticeably weaker. The Logit balance accuracy for Bitcoin-NASDAQ falls below 0.50 in the two longer intervals, 60 and 90 days, and averages 0.4997 across all four intervals, which is the lowest result at the signal level. This difference relative to the S&P 500 is economically justified, as the NASDAQ Composite is heavily weighted toward growth-sensitive technology stocks, whose volatility may be less closely tied to global risk appetite than that of the broader index. For precious metals and dollar-dependent currencies, the results are mixed but generally more likely to hold over short periods. This is consistent with the view that cryptocurrency movements contain some information about tensions in these markets, although the predictive signal is weaker.

4.11 Practical interpretation of signal-layer performance

The usefulness of the signal can only be assessed relative to a stated decision rule. An investor would consult it selectively, as one input to a conditional decision: whether to reduce exposure, tighten stops, or increase hedging when the stress probability rises.

The exit rate (how often the signal is active) is then as important as accuracy. A signal that never activates is useless whatever its nominal accuracy; one that activates constantly raises trading costs and stops carrying information. The logistic model sits between these extremes, active on roughly a fifth to a third of days depending on the pair (Table 6).

The most direct test of practical value compares the conventional asset's next-day return on days the signal flags as stress against days it leaves clear. Table 6 reports this for the logistic signal, averaged across the four window lengths.

Table 6: Average next-day return of the conventional asset on days flagged by the logistic signal versus days left clear, by asset pair (Logit, averaged over the four windows). Returns are in basis points per day; a positive difference means flagged days are, on average, worse. The flag rate is the share of days on which the signal is active.

Asset pair	Flagged (bp)	Clear (bp)	Difference	Flag rate (%)
BTC–S&P 500	4.08	5.05	+0.96	33.1
BTC–NASDAQ	4.70	6.64	+1.94	32.0
BTC–Gold	7.01	7.10	+0.08	20.1
BTC–Silver	10.23	10.13	−0.10	26.4
BTC–USD index	0.57	0.89	+0.32	31.4

This pattern is confirmed by the classification results. For the two equity pairs, the average next-day return is lower on flagged days than on clear days, by about 1.9 basis points per day for the NASDAQ and 1.0 basis point for the S&P 500. This means that the signal shifts exposure away from relatively worse days, which is its intended use. For gold and silver, the difference is negligible or slightly negative, which is consistent with the weak classification performance for these pairs.

The Dollar Index is a case apart. Its absolute difference is small, at +0.32 basis points, but the average daily return of the series itself is also very small. In relative terms, this makes the gap between flagged and clear days the largest in the table. Given how weakly the classifier performs on this pair, this result is better read as a small-denominator artifact than as evidence of a useful signal.

Two further points are important. First, the average flagged-day return is still positive because the sample period is predominantly a rising market. The signal therefore reduces exposure to relatively worse days rather than predicting outright losses. Second, these figures are before transaction costs, which a turnover of roughly one third of days could erode. Overall, the signal works as a modest risk-reduction overlay for broad-equity exposure, but it is not strong enough to stand alone as a timing rule.

4.12 Sensitivity to rolling window length

Window length is a first-order choice, and it visibly shapes both the target and the results. Short windows give responsive but noisy estimates; long windows give smooth, heavily autocorrelated targets that are easier to forecast but slower to register a real structural change.

The experiments identify the medium windows, and $w = 30$ days in particular, as the most useful compromise: smooth enough for high out-of-sample R^2 while still registering stress episodes without much delay. At longer windows the persistence effect strengthens further, narrowing the gap between the simple baselines and the complex models. This interaction between window length and model performance is itself a structural finding and a useful guide for selecting the horizon in future work.

4.13 Summary of empirical findings

The empirical results of the thesis are summarised in four principal statements:

1. Time-varying intermarket dependency between Bitcoin and conventional assets is forecastable in an out-of-sample walk-forward framework, with high average R^2 across every model class except the DCC-GARCH benchmark.
2. The dominant source of predictive power is the serial persistence of the rolling correlation target; Ridge, AR(1), and HAR sit at the top with practically identical accuracy, all exploiting this persistence, while tree-based methods trail substantially.
3. Machine-learning models have lower RMSE than the leakage-safe DCC-GARCH benchmark on the smoothed rolling-correlation target; Ridge achieves the best overall average RMSE (0.0656) across all 24 experiments. Because the reported DM standard errors do not accommodate dependence induced by the overlapping target, this is a descriptive performance comparison rather than a formal significance claim. As the discussion of this benchmark in Chapter 5 stresses, the comparison is structurally tilted toward the ML models, since the target is a smoothed rolling correlation while DCC-GARCH estimates the instantaneous conditional correlation.
4. The investor signal layer delivers modest classification results (on average slightly above chance, though not subjected to a formal significance test), strongest for broad equity markets. It is a supplementary risk-management input, not a self-contained directional predictor.

These findings are consistent with one another and, together, enough to carry both the scientific and the applied contributions of the thesis. The next section takes up what they mean more broadly.

The evidence gives a consistent picture of how crypto information enters conventional-asset risk. The relationship is real and measurable, but most of the predictive information comes from the persistence of the dependency series itself. The nonlinear ability of gradient-boosted and random-forest models adds little in this setting. The dependency changes slowly over time and is sensitive to different market conditions, so simple, low-complexity models work better than more flexible, high-capacity ones. Chapter 5 develops this interpretation and places it in the wider literature on intermarket contagion and financial machine learning.

5 Discussion

5.1 The core finding

The rolling intermarket dependency between Bitcoin and conventional assets can be predicted out of sample. The result is not an artefact of in-sample overfitting: the signal withstands strict temporal-ordering constraints for all six pairs and four window lengths. Predictability depends almost entirely on persistence, since the rolling correlation shows strong serial dependence, and its current value is largely determined by its recent past.

These two observations are consistent with each other: the dependency is predictable mainly because it is persistent. The same pattern appears again and again over time, which makes the dependency easier to predict.

5.2 Why simple models dominate

The fact that Ridge, AR(1), and HAR finish at almost the same level at the top of the ranking says something about the data-generating process. A rolling Pearson correlation computed over w days shares $w - 1$ observations with the previous window, which mechanically creates strong autocorrelation. After the Fisher transformation, the first-order lag captures the vast majority of the forecastable variance. In these circumstances any reasonable model relies heavily on the most recent value, and a model that does little else is difficult to surpass.

That Ridge exceeds AR(1) by only 0.0003 RMSE (0.0656 versus 0.0659) is not a crucial advantage of machine learning; it reflects the fact that regularised linear regression, when scaled correctly, restores the same autoregressive signal with a slightly richer feature set. HAR corresponds almost exactly to AR(1), using three time scales (daily, weekly, and monthly) that match the overlapping-window memory structure of the target. The convergence of three structurally different methods to almost the same accuracy indicates that the prediction problem has an almost linear autoregressive structure, and that none of these methods extracts significantly different information.

The pattern is well documented in financial forecasting [26]: signal-to-noise ratios in financial markets are low, and the additional information in auxiliary features is often dominated by the autoregressive component. In this study the pattern became apparent only after a long hyperparameter search over tree configurations had been carried out.

The much lower performance of the tree methods is still useful evidence. It is diagnostic: the fact that well-configured XGBoost and Random Forest models cannot match Ridge or AR(1) indicates that the dependency process contains little exploitable nonlinear structure at the one-step horizon, at least within the scope of the feature set used here.

5.3 ML compared with DCC-GARCH: a fairer comparison

Ridge and ElasticNet have lower RMSE than DCC-GARCH in the reported experiments, and the descriptive gap is large: an average RMSE of 0.066 for Ridge compared with 0.214 for DCC-GARCH, more than a factor of three. The reported DM statistics are positive, but their zero-lag HAC standard errors do not account for autocorrelation induced by the overlapping rolling target; the associated p-values are therefore not used as formal significance evidence.

The correct interpretation is not that DCC-GARCH is a bad model. DCC-GARCH is a consistent conditional covariance model, and it produces instantaneous conditional correlation, whereas the goal here is a smoothed moving average. Projecting the DCC-GARCH output onto this moving target results in a systematic lag that the model was never intended to eliminate. Machine learning models trained directly on the Fisher-transformed moving target do not suffer from this mismatch, as they are optimized from the outset to achieve the same target.

A second distinct issue is data harmonization. Since conventional data series are fed into Bitcoin's seven-day calendar, the GARCH components for the benchmark are estimated based on inverted series in which approximately 30% of the observations consist of forced zeros on weekends and holidays (Appendix A). This degrades the volatility estimates used in calculating the conditional correlation and further widens the gap with machine learning models. Re-estimating the DCC-GARCH model using only regular trading days would help identify this effect and is planned for future work. However, the robustness check for weekends in Appendix A shows that the constancy of the target itself—a property utilized by machine learning models—remains virtually unchanged under the calendar adjustment.

We should be honest about why there is a performance gap. If we simply say that machine learning is better than the econometric model without considering these differences, we make the results look stronger than they actually are.

5.4 Strengths and limitations of the investor signal

The signal layer gives moderate results in every dimension: AUC of approximately 0.48 to 0.56, balanced accuracy from below 0.50 to about 0.53 depending on the pair and window, and, for the logistic model, exit rates from 16 to 35%. None of these numbers is strong in itself. Together they describe a signal that, on average, classifies stress regimes somewhat better than chance, activates with a reasonable frequency, and carries more information about equity stress than about precious-metal or dollar-index stress.

The equity result is amenable to economic interpretation. Bitcoin and the S&P 500 share exposure to global risk appetite, funding liquidity, and sentiment cycles. When the dependency between them is rising or shifting, this often reflects broader market-regime transitions that also increase equity downside risk. The cryptocurrency feature set is reasonably well specified for that channel.

Gold and silver respond to a broader range of factors, including real interest rates, inflation expectations, geopolitical risks, and demand for safe-haven assets. Bitcoin's price movements do not reflect these factors very well. Thus, the weaker signal for these assets reinforces the same idea: the set of features captures some of the factors causing market stress, but not all of them.

In practice the signal belongs in the toolbox as an auxiliary risk indicator; used as a standalone alarm it would disappoint. Its natural place is next to fundamental valuation, macro analysis, and position-sizing rules that raise the bar for accepting risk when the dependency-based probability of a stress regime is elevated.

5.5 Methodological choices that mattered

Several design decisions were important. The most important was running DCC-GARCH through the same expanding-window walk-forward as every other model: a full-sample DCC fit would have given the benchmark access to future data and made the comparison invalid. The Fisher transformation mattered because the bounded Pearson correlation distorts errors close to ± 1; the transformed target is closer to Gaussian, and RMSE becomes a sensible loss over the entire range. Reporting all four window lengths allowed the study to show honestly how much predictability depends on the autocorrelation structure, which the progression from $w = 14$ to $w = 90$ makes visible. Finally, reporting several metrics and paired DM diagnostics showed that DCC-GARCH has higher average error in every reported pair and window; formal inference requires a HAC specification that accounts for the rolling-window overlap.

5.6 Statements this thesis does not make

Several claims are explicitly not being made.

Bitcoin does not predict equity returns. It partially predicts the evolving dependency between Bitcoin and equities, which is a weaker and more precise statement. The signal layer estimates the probability of increased downside risk for conventional assets; it says nothing about the direction, size, or timing of the movement within a flagged period.

The results are based on predicting one step ahead. Multi-step performance may differ significantly, especially for machine learning, where the autoregressive advantage weakens as the horizon expands. This is an open question.

These results are based on one-step-ahead forecasting. Multi-step performance may vary significantly, especially in the case of machine learning, where the advantage of autoregression diminishes as the forecasting horizon increases. This question remains open.

5.7 Iterative model development: what changed across versions

The final model presented in this thesis is the result of several months of iterative refinement. This section documents that process, because the sequence of changes is itself an empirical result about the nature of the forecasting problem.

Version 1 (September 2025): minimal baseline. The first working prototype had seven models: the last-value baseline, AR(1), Ridge, ElasticNet, Random Forest, Gradient Boosting, and XGBoost. Feature engineering was very simple. We used four lags of the rolling correlation, added short-window volatilities for both assets, and included lagged returns up to lag 5. We did not include a DCC-GARCH benchmark at this stage.

Hyperparameter tuning was done ad hoc. Ridge used alpha = 1.0, ElasticNet used alpha = 0.01, and XGBoost used 1000 estimators with a learning rate of 0.05. We did not scale the features for the linear models.

For BTC-USD versus S&P 500 at $w = 30$, the best out-of-sample RMSE was 0.0897 for Ridge. The AR(1) model performed comparably.

Version 2 (November 2025): DCC-GARCH and signal layer. The second major milestone introduced the DCC-GARCH econometric benchmark via a walk-forward protocol that matched the expanding-window setup used for machine-learning models. This was a critical correctness fix: an earlier full-sample DCC fit would have granted the benchmark access to future data, invalidating the comparison. The investor signal layer was added as a second stage, classifying whether conventional assets were entering stress regimes using the rolling-dependency features. Core model hyperparameters and feature sets were unchanged from Version 1.

Version 3 (February 2026): stabilisation. In this version, the pipeline was stabilized as a reliability baseline. There were no improvements in metrics; the primary focus was on reliability. We added a fallback version of XGBoost from GPU to CPU, improved exception handling, and made metric export consistent. Out-of-sample results were nearly identical to those of Version 2, confirming that the performance degradation in Version 1 was not caused by implementation errors.

Version 4 (May 2026): feature expansion, regularisation revision, and HAR. The final version incorporated several targeted improvements informed by the Version 1–3 plateau:

- Feature engineering expanded from ≈15 to ≈35 features. New features include momentum and z-score of the rolling correlation (dep_mom_5, dep_mom_20, dep_zscore), extended lags (dep_lag20, dep_lag60), a 60-day volatility window, the volatility ratio between assets, squared-return proxies, and a short-run correlation momentum term. The Heterogeneous AutoRegressive (HAR) model [13] was introduced as a theoretically grounded alternative to AR(1). HAR uses three temporal scales simultaneously, fitted on the Fisher-z target exactly as in Eq. (7): $\hat{z}_{t+1} = \phi_0 + \phi_d\, z_t + \phi_w\, \bar{z}_t^{(5)} + \phi_m\, \bar{z}_t^{(22)} + \varepsilon_{t+1}$, where $\bar{z}_t^{(k)}$ denotes the equally-weighted average of the previous k observations. Originally developed for realised variance, HAR is well-suited to rolling correlations because the overlapping-window structure creates natural heterogeneous memory at daily, weekly, and monthly horizons. Ridge's penalty strength (scikit-learn's alpha, the λ of Eq. (8)) was reduced from alpha = 1.0 to alpha = 0.5 and an explicit StandardScaler was added. The combined effect was dramatic: Ridge RMSE at BTC-USD vs S&P 500 $w = 30$ improved from 0.0897 (Version 1) to 0.0651, nearly matching AR(1). The cross-validated optimum actually sits even lower (around alpha = 0.001; see Figure 16), but the CV curve is essentially flat across this range, so the slightly firmer alpha = 0.5 was kept for a more stable fit at negligible cost in RMSE. Conversely, XGBoost regularisation was tightened (learning rate 0.05 → 0.02, added L_1/L_2 penalties), reflecting the insight that the expanded feature set increased overfitting risk for tree-based methods. The walk-forward ensemble now weights each constituent model by the inverse of its RMSE over the most recent 60 out-of-sample predictions, instead of equal weights. This allows the ensemble to adapt dynamically to regime shifts in which different models temporarily dominate.
- Bootstrap confidence intervals and sensitivity analyses. 500-replicate bootstrap CIs were added for RMSE and R^2, and a refit-frequency sensitivity sweep across $\{5, 10, 20, 42, 63\}$ days (which includes the default of 20) was added to verify that $refit_every = 20$ is not a cherry-picked choice.

The lesson of the four versions. The biggest lesson from the four model versions is that adding more variables did not improve the tree-based models. Increasing the feature set from 15 to 35 variables did not make XGBoost better. Its RMSE stayed almost the same or became slightly worse after the expansion. On the other hand, Ridge improved substantially because it uses the full feature set in a linear way with regularisation. This result fits with the main point of the thesis. The forecasting target has a structure that is almost linear and autoregressive. The additional information from volatility ratios, momentum, and return squares is useful for the regularised linear model, but it does not give gradient-boosted trees useful nonlinear patterns to exploit.

For future work, this means that even more sophisticated features are unlikely to bridge the gap that still exists between AR(1) and machine learning in solving this problem. Real progress will most likely be achieved through a different approach to modeling, such as the use of multi-stage horizons, high-frequency data, or the simultaneous modeling of multiple correlation pairs.

5.8 Limitations

The main limitations are: daily frequency (no intraday), one-step horizon, Pearson correlation as the only dependency measure, a single base cryptocurrency (Bitcoin), and no portfolio backtest. Each represents a natural extension that could sharpen or complicate the findings. None of them invalidates the core empirical results within the scope of what was actually tested.

6 Conclusion

This thesis examined the following questions: Does information about the relationship between Bitcoin and conventional assets help predict how that relationship will change, and can these predictions aid in risk management? The answer to both questions is yes. However, it is necessary to clarify what "useful" means.

The relationship between assets can be predicted based on a sample. This conclusion holds true for six asset pairs, four different time intervals, and several model types tested using a step-by-step procedure. Longer intervals make the trend smoother and more stable over time, leading to more accurate forecasts. With a 90-day interval, even a simple AR(1) model yields an R^2 ranging from 0.982 to 0.992 across all six pairs.

This finding also points to an important limitation: the forecast is largely based on constancy. Since the 30-day moving correlation matches the previous day's data in 29 out of 30 observations, it is difficult to make a forecast that would be any better than simply saying that tomorrow's correlation will be the same as today's. The Ridge, AR(1), and HAR models fall within the range of statistical noise, and the simple last-value forecast is just behind them. The Random Forest, GBM, and XGBoost models perform worse. Tree-based models do not outperform the baseline autoregressive model, while Ridge performs at a similar level. All machine learning models perform better than the DCC-GARCH test.

This last comparison warrants careful consideration. The target metric is the moving Pearson correlation, whereas DCC-GARCH estimates the current conditional correlation. Comparing the DCC-GARCH output to the target metric introduces a lag that does not align with the benchmark. Thus, the main takeaway is limited: flexible models fit this specific objective more accurately than a parametric model, but they do not perform better than a simple stationary model. This does not mean that DCC-GARCH generally performs worse than a market-dependent model.

The removal of feature groups in Section 4.6 illustrates the same idea from another perspective. A single lag in the dependent variable already accounts for most of the variation that can be predicted. Adding more historical data to the dependent variable yields only a slight improvement. Adding groups of interrelated objects does not improve the linear model and actually worsens the tree-based models. Tree-based models perform worse as more groups of objects are added.

The investor signal level is more complex. The balanced accuracy ranges from 0.50 to 0.53, while the AUC ranges from 0.48 to 0.56. These values are modest. The probability of exceeding them is high for most, but not all, configurations. The probability of a decline in the Bitcoin–NASDAQ pair is lower over longer time frames. The strongest signal is observed for the Bitcoin–S&P 500 pair, where the characteristics of the cryptocurrency are more closely linked to factors affecting the broader stock market. For gold, silver, and the dollar index, the signal is weaker. This weak signal makes sense because these markets are influenced by factors that Bitcoin's dynamics do not adequately account for.

From a practical standpoint, this signal should be used as part of a broader risk management process. If the signal were used on its own as a warning, its importance would be overstated. The signal does not predict the next day's returns. Instead, it provides a small and relatively consistent shift toward caution as the structure of dependencies

changes. This is a limited but reasonable use of the signal.

Future work should test the system over longer forecasting horizons, where the benefits of consistency may not be as evident. Future work should also consider a broader set of characteristics, including chain-linked indicators and macroeconomic variables, and compare the results with dependent indicators that capture extreme movements in joint variances more accurately than Pearson's moving correlation. A portfolio test that incorporates transaction costs will help make the results more realistic. These ideas are left for future work, and this thesis serves as a foundation for them.

Appendix A Data Description and Additional Background

This appendix collects the material that would have made the main text slower to read. It contains the background and diagnostics that support the empirical setup, allowing the main body to remain focused on the argument and its interpretation.

A.1 The descriptive role of the asset universe

The assets were not selected for this dissertation at random. Each instrument represents a distinct part of the traditional financial system, which makes it possible to compare these different parts directly with Bitcoin. Bitcoin is the main object of comparison and is included in every asset pair.

- The S&P 500 is used as a broad measure of U.S. equity-market risk and general investor sentiment.
- The NASDAQ Composite represents a more growth-oriented and technology-heavy part of the market, which makes it particularly relevant when studying crypto assets.
- Gold and silver ETFs represent exposure to precious metals, a market area often linked to hedging, inflation expectations, and safe-haven discussions.
- The U.S. Dollar Index proxy represents defensive positioning and changes in the strength of the U.S. dollar in global markets.
- Ethereum is included as a crypto-native comparison asset, allowing same-ecosystem dependence to be compared with dependence across different markets.

These assets differ in terms of their investor base, liquidity, sensitivity to broader economic conditions, and importance within the financial system. This variety helps reduce the risk that the conclusions are driven by only one narrow part of the market.

A.2 Visual overview of the dataset

Figure 6 shows the normalised price paths for the asset universe. Normalisation sets all series to the same starting point, making differences in the underlying market paths easier to see.

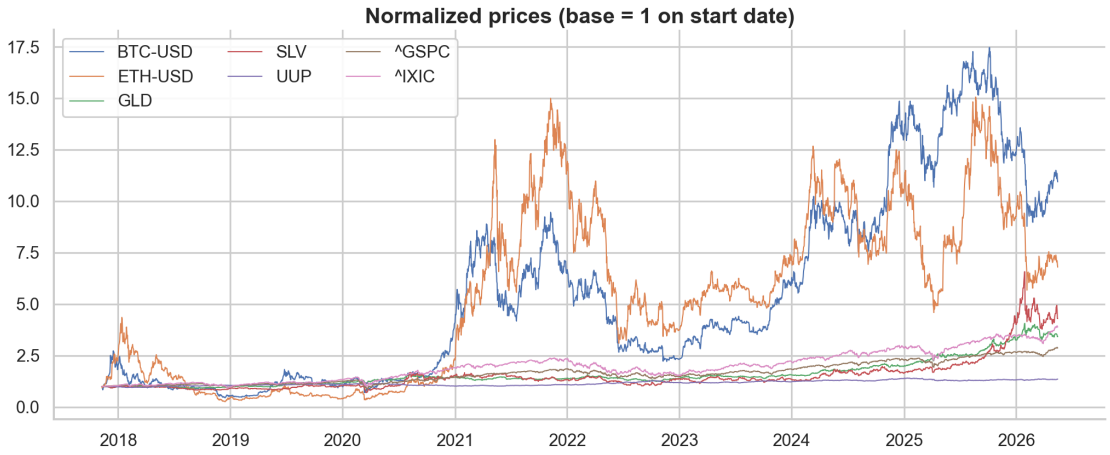


Figure 6: Normalised prices of the assets included in the study.

Figure 7 shows the rolling volatility profiles. The profiles show that Bitcoin and the other crypto assets operate in a considerably more volatile environment than the traditional assets. This is another reason for examining dependence as a time-varying process.

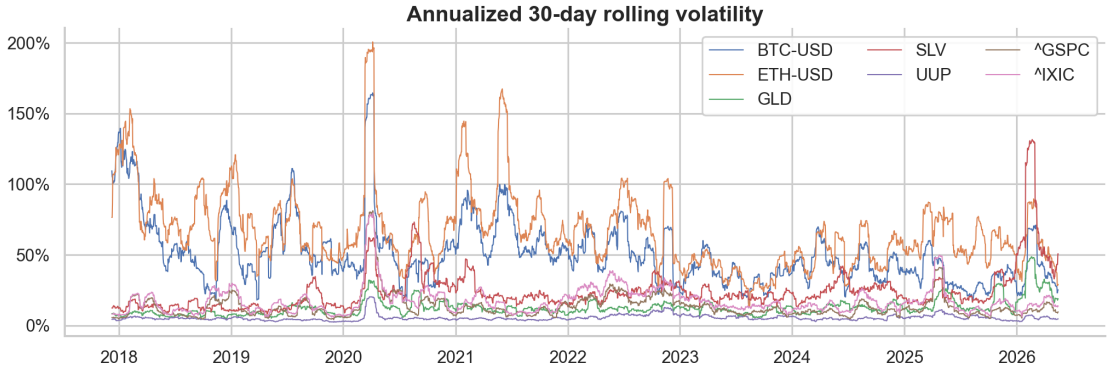


Figure 7: Rolling volatility across the asset universe.

Figure 8 provides the static correlation heatmap, while the dynamic rolling-correlation structure is shown in the methodology chapter (Figure 1). Taken together, the two figures support the main modelling choice of the thesis: static correlation measures are not sufficient to describe the cross-market relationships studied here.

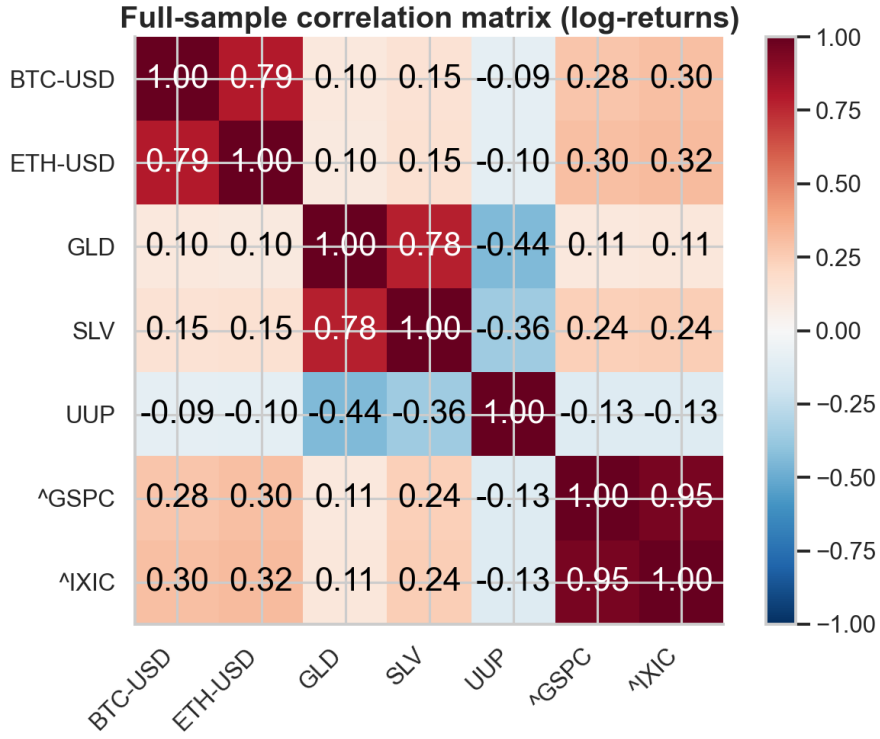


Figure 8: Static correlation heatmap of the return series.

A.3 Dataset validation and quality diagnostics

All price series are checked systematically before any modelling step. The diagnostics cover four areas: observation coverage, missing-value frequency, extreme-return events, and distributional shape. Table 7 summarises the results for each ticker.

Table 7: Data quality summary per ticker

Ticker	First obs.	Last obs.	N	% miss.	Gaps >2d	Zero ret.	Skew	Ex.Kurt	Ann.Vol (%)
BTC-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.73	13.48	55.88
ETH-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.74	10.86	72.40
GLD	2017-11-09	2026-05-16	3053	0.00	0	920	-0.87	13.63	13.88
SLV	2017-11-09	2026-05-16	3053	0.00	0	955	-2.69	49.30	28.49
UUP	2017-11-09	2026-05-16	3053	0.00	0	1001	-0.03	8.56	5.84
^GSPC	2017-11-09	2026-05-16	3053	0.00	0	914	-0.74	22.67	16.15
^IXIC	2017-11-09	2026-05-16	3053	0.00	0	915	-0.46	12.66	19.62

Figure 9 confirms that all seven tickers share an uninterrupted observation window from November 2017 to May 2026, with no gaps in coverage after alignment.

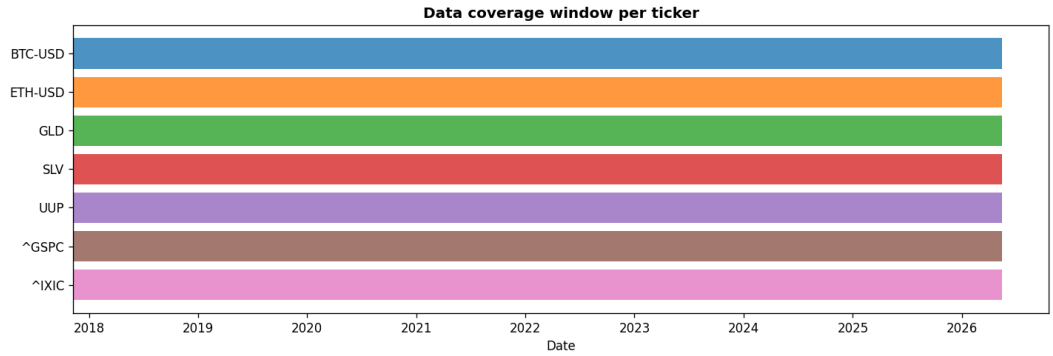


Figure 9: Observation coverage window per ticker. All assets share the aligned date range after the ETH-USD availability cutoff in November 2017.

Figure 10 shows instances of extreme returns ($|z\text{-score}| > 5\sigma$) for each asset. Bitcoin and Ethereum exhibit outliers with the largest absolute values. This is consistent with the well-known behavior of cryptocurrency returns, with an excess kurtosis of 13.5 for Bitcoin. However, Bitcoin and Ethereum are not the most leptokurtic series in the sample. Silver has an excess kurtosis of 49.3, while the S&P 500 index has an excess kurtosis of 22.7. These values are not directly comparable to the cryptocurrency series, since the moments for the five conventional assets are calculated using return vectors in which approximately 30% of observations are zero on weekends. This reduces the variance and increases the standardized fourth moment. Thus, all five traditional assets contain a large number of days with zero returns. This is not an ETF-specific issue. The problem arises because the assets trade five days a week, in contrast to Bitcoin's seven-day trading calendar. The intervals between weekends are filled with zeros, meaning that every Saturday and Sunday, returns are forcibly set to zero, while Bitcoin continues to be observed because it trades continuously. This calendar discrepancy is a significant limitation. On weekends, Bitcoin's actual return is matched against the zero return of a conventional asset, and this mechanical matching reduces the moving correlation. Another issue is that forward-filling may increase the persistence of the target. Persistence matters because it is an important part of the results described in Chapters 4.1 and 5.

To test this directly, the 30-day moving correlation between Bitcoin and the S&P 500 was recalculated using only regular trading days. Rows corresponding to 914 weekends and holidays were removed, and the persistence of this version was compared to that of the version using the full calendar. The persistence measure remained virtually unchanged. In fact, when weekends are excluded, the persistence measure is slightly higher. The autocorrelation with a lag of 1 for the target Fisher-z measure is 0.977 for the seven-day calendar and 0.981 when considering only trading days. The naive "tomorrow equals today" coefficient R^2 is 0.954 compared to 0.961. The same pattern is observed for the 14-day and 90-day periods.

Thus, forcing weekends to zero introduces some noise rather than artificial persistence. The dominant role of persistence is not due to the calendar convention. Limiting the analysis to regular trading days would be a useful refinement for future work, but it would not alter the main conclusions. None of the detected anomalies are data errors. The anomalies reflect real price changes and are therefore retained in the analysis.

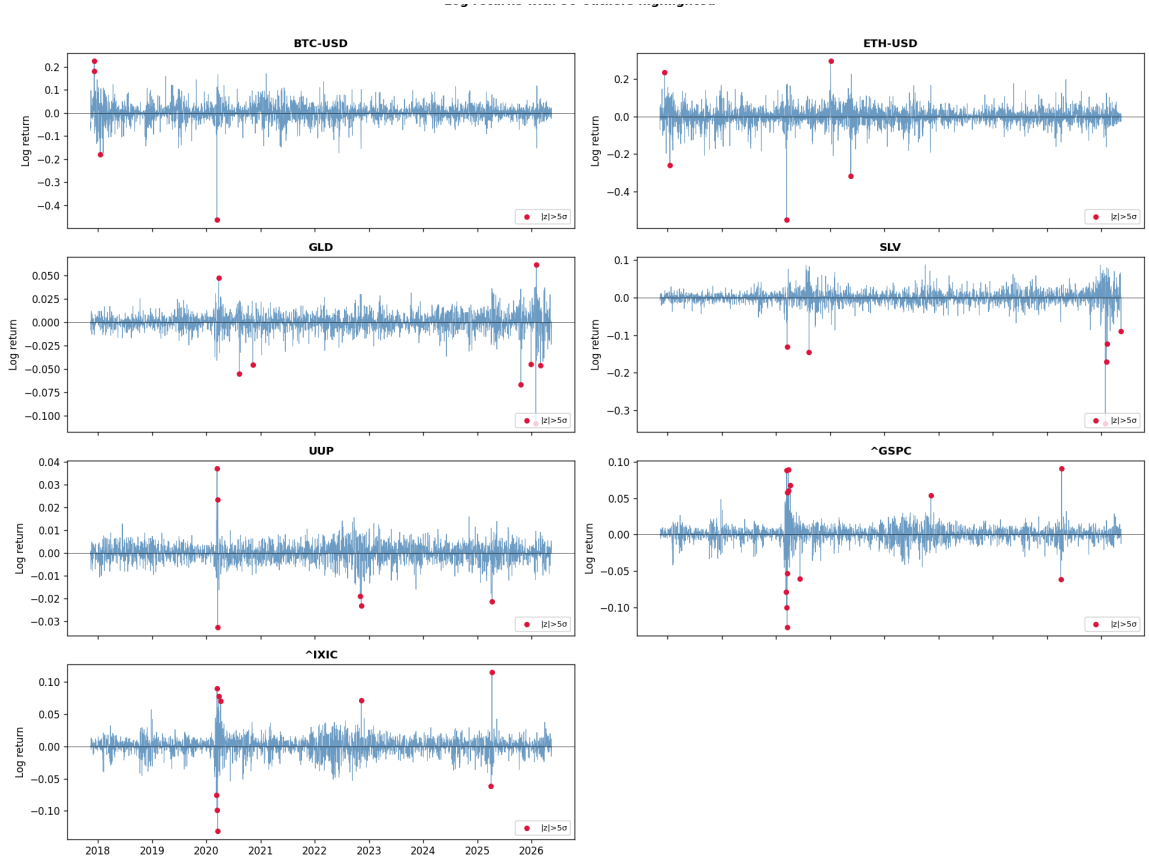


Figure 10: Log-return series with extreme observations ($|z| > 5\sigma$) highlighted in red. Crypto assets contain occasional large outliers, while conventional assets are more contained but are not free of extreme observations.

A.4 Fisher-z transformation of the correlation target

The rolling Pearson correlation ρ_t is bounded between -1 and 1 and has compressed variance near the boundaries. Applying the Fisher-z transformation $z_t = \tanh^{-1}(\rho_t)$ maps the target onto the real line and makes the variance more stable across the range of correlations.

Figure 11 illustrates the distributional improvement for the representative Bitcoin–S&P 500 pair at the 30-day window. The untransformed correlation distribution is left-skewed and bounded, while the Fisher-transformed version is more symmetric and approximately Gaussian.

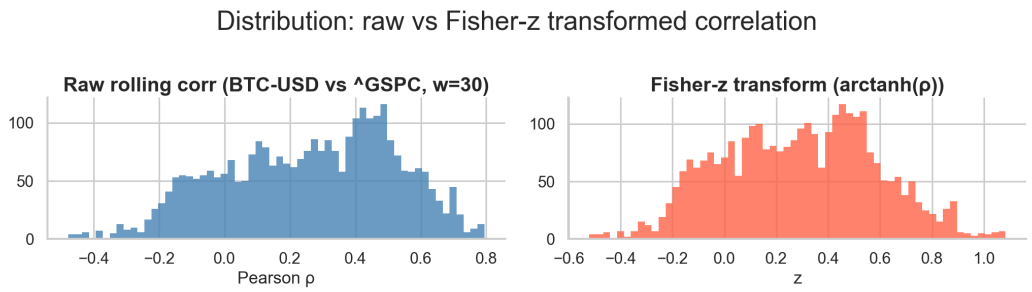


Figure 11: Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and reduces boundary compression.

A.5 Why multiple rolling windows are used

The thesis uses 14-, 30-, 60-, and 90-day windows because no single time horizon is appropriate for every purpose. A short window reacts quickly to recent shocks but is noisier. A long window is smoother and more persistent but slower to capture a change in the underlying regime. Using all four windows makes it possible to examine these differences.

The choice of window length also shapes the overall forecastability profile. Longer windows are easier to forecast because they are smoother, but this forecasting success is largely explained by persistence. Shorter windows are noisier, but they can provide more information about local instability.

A.6 Additional implementation remarks

Each time the pipeline is run, the agreed results are written to the `outputs/results`, `outputs/tables`, `outputs/figures`, and `outputs/predictions` folders. This supports reproducibility and also makes the reporting process easier. Since the tables and figures are generated directly from the code, the gap between running the experiment and preparing the report is kept small.

The notebooks were developed in line with the main development process and were not allowed to drift away from it. The figures produced in the notebooks remain interpretable, the control metrics are selected consistently, and helper functions are reused without relying on fragile path assumptions.

This is important in practice because research notebooks can easily diverge from the code used to produce the results and become unreliable over time.

A.7 Summary

The conclusion is mainly methodological: the environment studied here is heterogeneous, economically non-stationary, and sufficiently complex to justify treating the relationship as time-varying. The descriptive evidence supports the modelling strategy presented in the main text.

Appendix B Supplementary Forecasting Results

This appendix gathers the extra forecasting diagnostics and model-comparison plots that complement the main discussion. It leans on figures rather than tables: changes in model behaviour come through far more clearly in a picture than in a very wide table.

B.1 Model-comparison figures

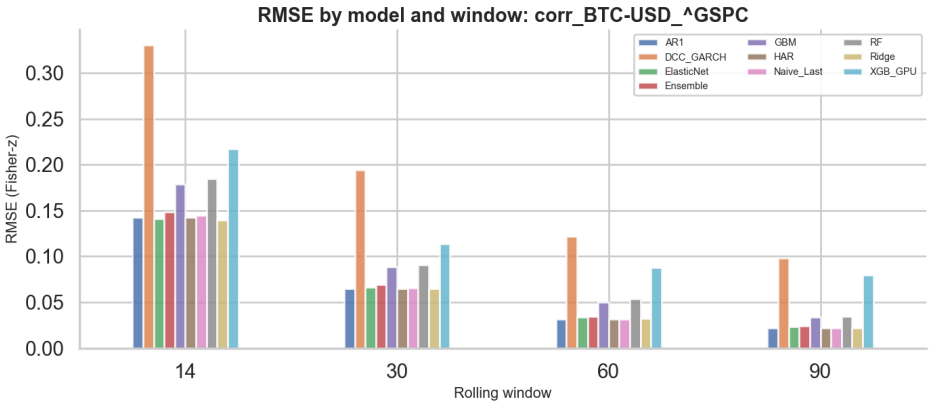


Figure 12: RMSE by model and window for the representative Bitcoin–S&P 500 pair.

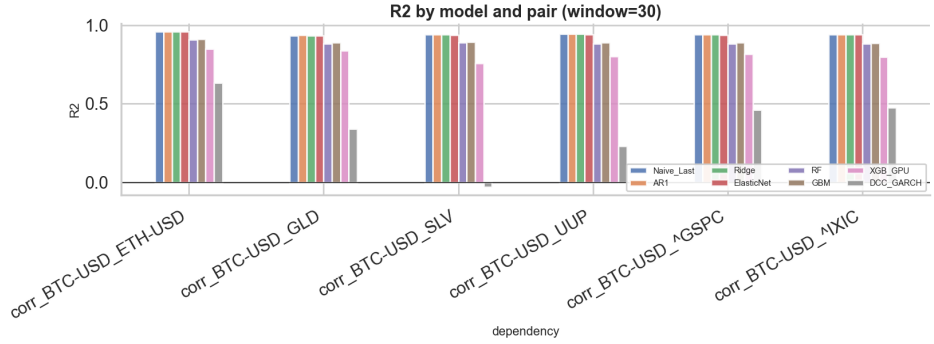


Figure 13: Model R^2 comparison for the 30-day window.

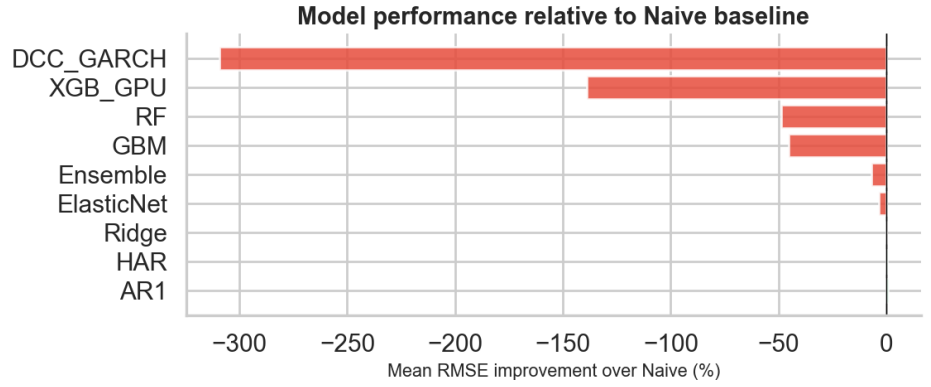


Figure 14: Improvement relative to the naive baseline.

B.2 Hyperparameter and feature diagnostics

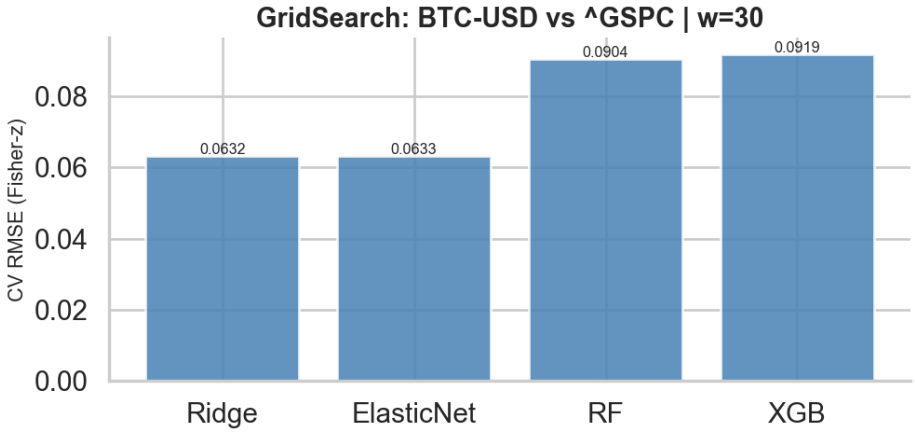


Figure 15: Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task.

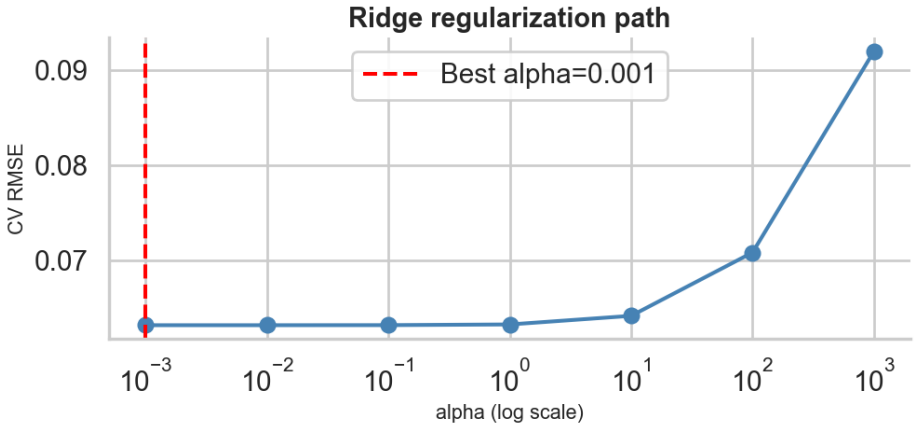


Figure 16: Regularisation path for the Ridge-based forecasting specification.

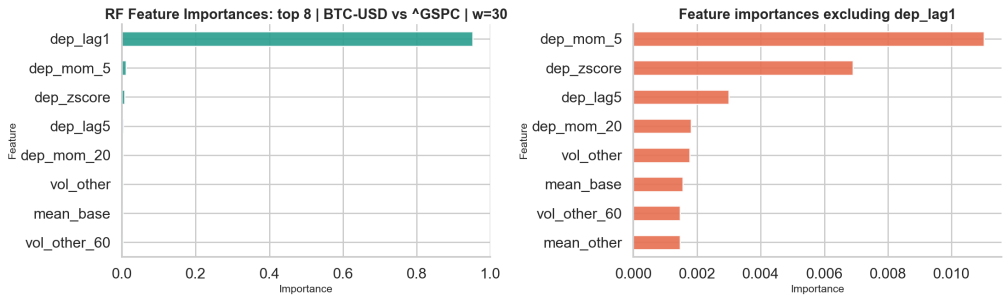


Figure 17: Random Forest feature importances for the Bitcoin–S&P 500 pair at the 30-day window.

B.3 Diagnostic plots for the representative equity pair

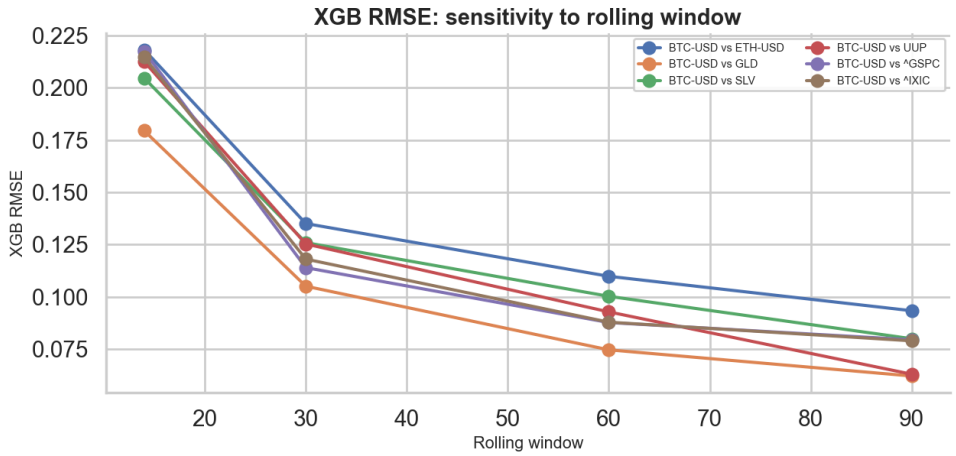


Figure 18: Window sensitivity of the XGBoost-based forecasting layer.

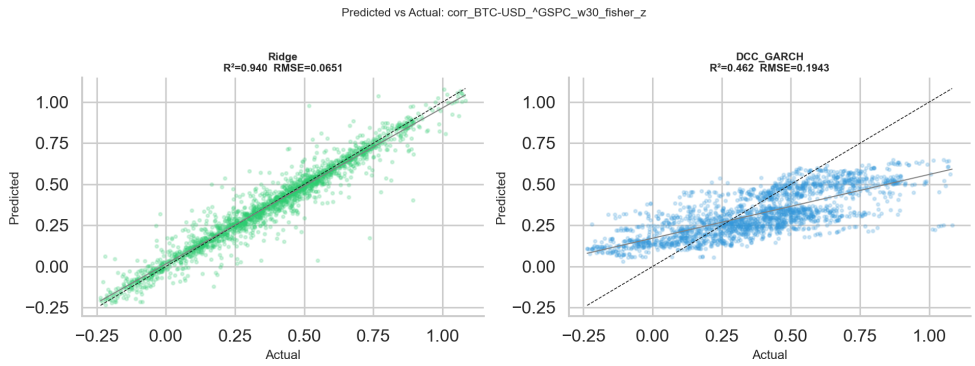


Figure 19: Forecast scatter diagnostic for the Bitcoin–S&P 500 dependency experiment.

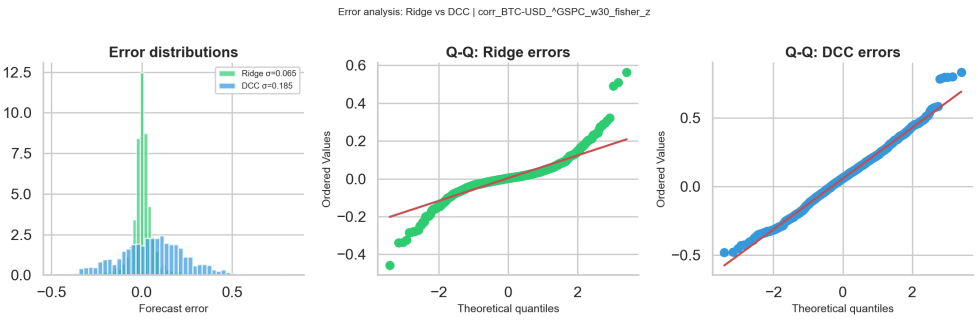


Figure 20: Forecast error distribution for the representative Bitcoin–S&P 500 dependency experiment.

B.4 Sensitivity by asset pair

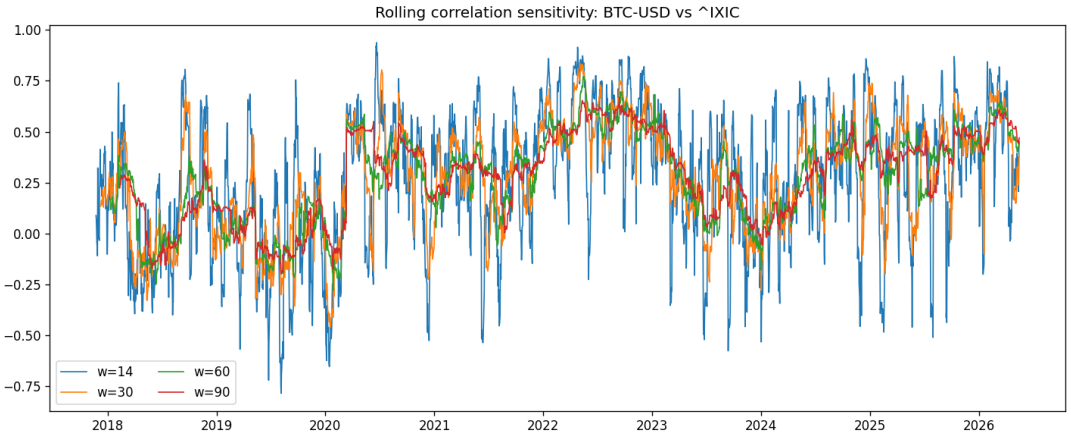


Figure 21: Sensitivity analysis for the Bitcoin–NASDAQ pair.

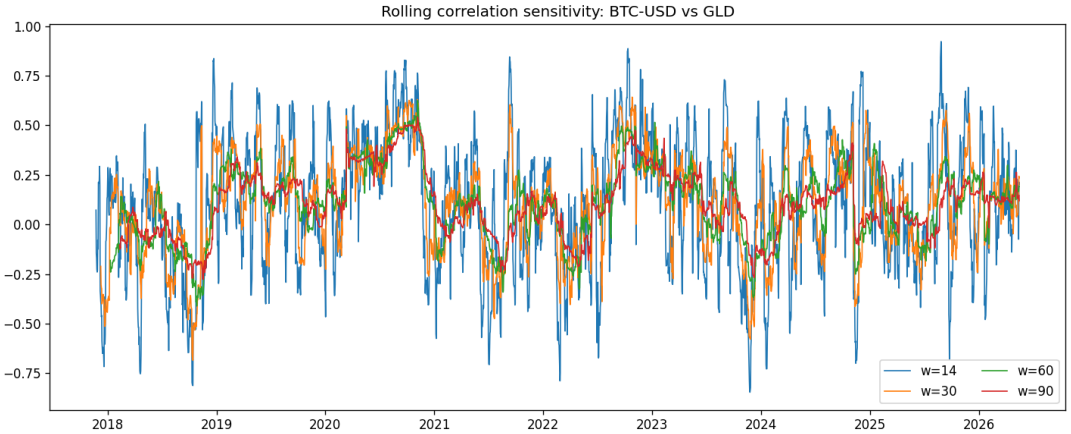


Figure 22: Sensitivity analysis for the Bitcoin–gold pair.

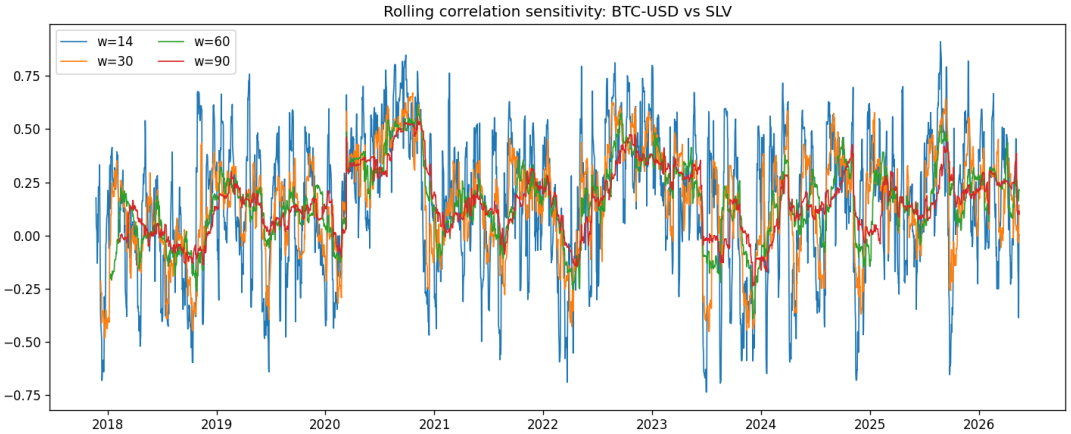


Figure 23: Sensitivity analysis for the Bitcoin–silver pair.

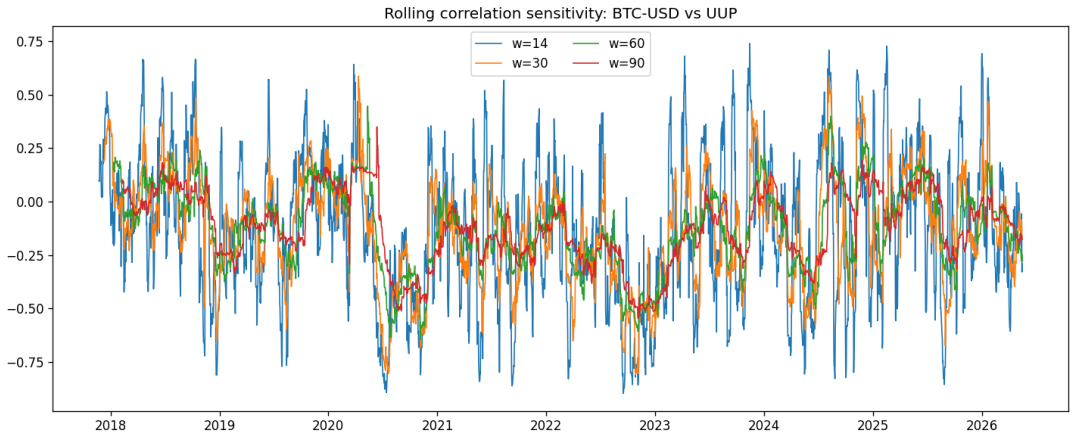


Figure 24: Sensitivity analysis for the Bitcoin–UUP pair.

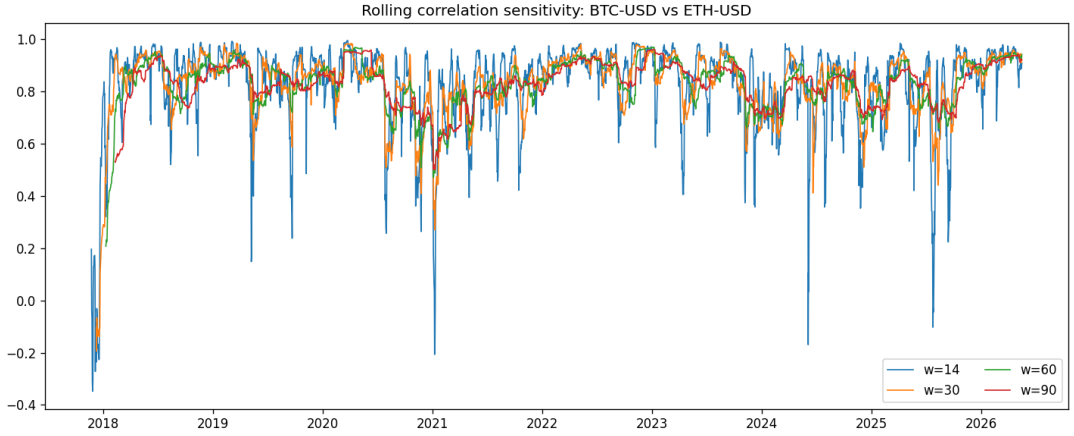


Figure 25: Sensitivity analysis for the Bitcoin–Ethereum pair.

B.5 Interpretation

Taken together, these figures support the main text. Forecast quality depends strongly on the choice of rolling window, while the target remains largely driven by persistence. The differences between model families are also clearer in a comparative plot than in any single ranking number.

Appendix C Supplementary Signal-Layer Results

This appendix gathers the supporting material for the investor warning layer. A representative signal output is shown below, and Table 8 gives the per-pair, per-window summary. The full set of signal figures for all pairs and windows is produced by the pipeline and is included with the repository.

C.1 Representative signal output

Figure 26 shows the signal for the Bitcoin–NASDAQ pair at the 30-day window, which is the weakest among the equity pairs. This differs from the stronger Bitcoin–S&P 500 output shown in the main text (Figure 5).

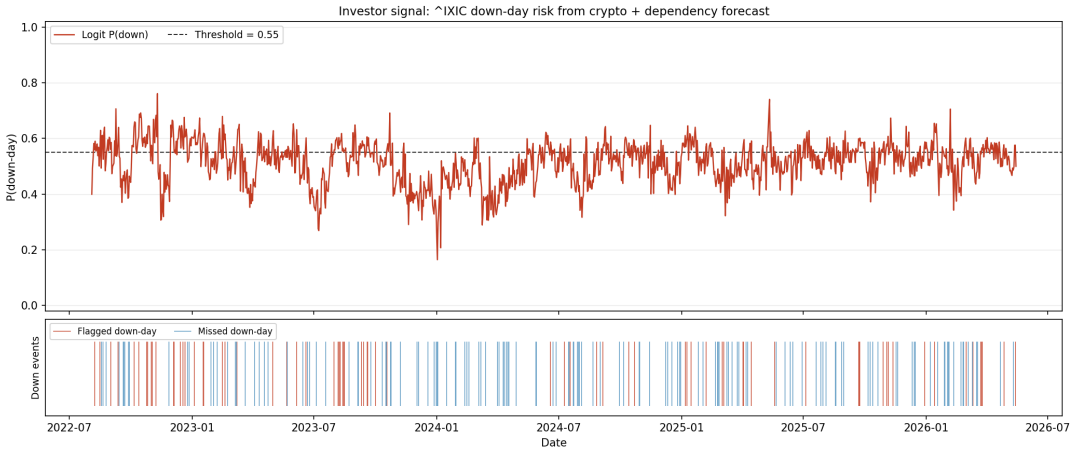


Figure 26: Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window. The upper panel shows the predicted stress probability and the decision threshold; the lower panel marks flagged and missed stress events.

C.2 Cross-market warning interpretation

The per-pair results show that the warning layer behaves differently across markets. The S&P 500 produces the clearest warning episodes, while NASDAQ is much weaker and falls below chance at the longer windows. This may be because its sector concentration adds idiosyncratic noise that the crypto channel cannot capture. The metal and dollar targets are noisier but are not uniformly uninformative. The signal should therefore be applied selectively rather than across the whole asset universe.

C.3 Summary table

Table 8: Balanced accuracy of the Logit signal layer by asset pair and rolling-window length. Family-level averages across all pairs and windows are reported in Table 5; this breakdown shows where the signal is informative.

Asset pair	w=14	w=30	w=60	w=90	Mean
BTC–S&P 500	0.5169	0.5308	0.5212	0.5264	0.5238
BTC–NASDAQ	0.5190	0.5016	0.4865	0.4917	0.4997
BTC–Gold	0.5053	0.5041	0.5078	0.5202	0.5093
BTC–Silver	0.4938	0.5187	0.5053	0.5189	0.5092
BTC–USD index	0.5071	0.5197	0.5131	0.5204	0.5151

This breakdown supports the main-text reading. The Bitcoin–S&P 500 pair is the strongest and most consistent, reaching 0.5308 at the 30-day window. Bitcoin–NASDAQ is the weakest, averaging 0.4997 and falling below the 0.50 chance level at the 60- and 90-day windows. The precious-metal and dollar pairs are positive on average but remain close to chance. The signal is therefore best applied selectively to broad-equity pairs rather than uniformly across the asset universe.

Appendix D Reproducibility and Code Structure

D.1 Project organisation

The computational section of the dissertation is organized more as a research project than as a set of ad hoc scripts. The main directories are laid out as follows:

Listing 5: Top-level project directory structure.

```
master-thesis/
|-- main.py                  # main pipeline entry point
|-- run_all.py               # full automation: pipeline + notebooks +
    ↪ LaTeX
|-- config.yaml              # experiment configuration
|-- requirements.txt         # Python dependencies
|-- thesis_app/              # core Python modules
|   |-- pipeline.py          # data, features, walk-forward, metrics
|   |-- dcc.py               # DCC-GARCH(1,1) likelihood + GARCH recursion
|   |-- dcc_walk.py          # leakage-safe walk-forward wrapper (uses dcc.
    ↪ py)
|   |-- signal_layer.py      # investor stress-warning layer
|   |-- notebook_helpers.py  # shared plotting and styling
|   |-- data_quality.py      # missing-value and outlier diagnostics
|   `-- regime_analysis.py   # historical regime catalogue
|-- notebooks/               # nine Jupyter notebooks (01-09)
|-- data/raw/                # cached downloaded price data
|-- data/processed/          # derived returns
|-- outputs/results/         # CSV summaries of experiments
|-- outputs/tables/          # LaTeX-exported tables
|-- outputs/figures/         # all generated figures
`-- thesis/                  # this document
```

D.2 Main implementation modules

The following modules constitute the core of the forecasting system:

- `pipeline.py`: orchestrates the full dependency-forecasting and signal-generation workflow.
- `dcc.py`: implements the core DCC-GARCH(1,1) log-likelihood and the one-step GARCH variance recursion used by the benchmark.
- `dcc_walk.py`: contains the leakage-safe walk-forward DCC-GARCH implementation, built on the routines in `dcc.py`.
- `signal_layer.py`: defines the investor warning layer and associated classification metrics.
- `notebook_helpers.py`: shared plotting helpers, consistent styling, and analysis utilities used across all notebooks.
- `data_quality.py`: performs missing-value, outlier, and coverage diagnostics on the raw dataset.

- `regime_analysis.py`: implements the historical regime catalogue and regime-conditional statistics.

This division of labor offers two benefits: it simplifies the audit of the workflow, and it allows any module to be tested or improved without affecting the others.

D.3 Software environment

The implementation is in Python, and the exact package versions are fixed in `requirements.txt`. The regularised linear models, tree ensembles, logistic classifier, and walk-forward splitting utilities come from scikit-learn [27]. GPU-based gradient boosting uses XGBoost [28]. The univariate GARCH(1,1) models used for the DCC benchmark are fitted with the arch package [29]. Market data are retrieved through yfinance [20]. The thesis is typeset using the finthesis class from the finlatex project [30], which provides the university title pages and document layout. Reproducibility is supported by the fixed random seed (`random_state = 42`, also used for XGBoost) and the frozen return file described earlier. No step depends on system time or on an unpinned remote source.

D.4 Analysis notebooks

Nine Jupyter notebooks in `notebooks/` supplement the main pipeline with interactive diagnostics, publication-quality figures, and robustness checks. Each notebook is self-contained and can be run either from the command line or interactively in Jupyter.

1. **`01_EDA_Dataset.ipynb`** — Exploratory data analysis. The notebook produces normalised price trajectories, rolling volatility profiles, a pairwise correlation heatmap, rolling pairwise correlation time series, and a Fisher-transformation comparison plot. These figures are presented in Appendix A.

2. **`02_GridSearch.ipynb`** — Hyperparameter search diagnostics. The notebook reports time-series cross-validation using an expanding-window `TimeSeriesSplit`, RMSE surfaces for the representative Bitcoin–S&P 500 experiment, regularisation-path plots for Ridge and ElasticNet, and Random Forest feature-importance charts. These results support the model-selection discussion in Chapter 3.

3. **`03_Model_Comparison.ipynb`** — Model-ranking summary. The notebook produces an average RMSE comparison bar chart, an R^2 comparison across window lengths, and an improvement-over-naive baseline figure for all asset pairs. These figures provide a visual complement to Table 3 in Chapter 4.

4. **`04_DM_Tests_Visuals.ipynb`** — Diebold–Mariano test visualisation. The notebook produces the DM heatmap comparing the best machine-learning model with DCC-GARCH (Figure 3) and the representative out-of-sample forecast panel for the Bitcoin–S&P 500 pair (Figure 2).

5. **`05_XGB_vs_DCC.ipynb`** — Error-profile diagnostics. The notebook produces the rolling RMSE time series (Figure 4), forecast-error scatter plots, and error-distribution comparison histograms for the representative experiment.

6. **`06_Regime_Analysis.ipynb`** — Regime-conditional analysis. The notebook produces the regime catalogue statistics table, regime-conditional RMSE bar charts,

and data-quality summary figures. These outputs provide the empirical basis for the regime-sensitivity discussion in Chapter 4.

7. **07_Robustness_Checks.ipynb** — Robustness and sensitivity analyses. The notebook contains refit-frequency sensitivity sweeps, stress-threshold sensitivity curves, bootstrap confidence intervals for key metrics, signal-layer diagnostics, and a cross-asset Diebold–Mariano test comparing equity and non-equity pair performance.

8. **08_Market_Events_Showcase.ipynb** — Landmark market-event case studies. The notebook covers eleven episodes between 2020 and 2026, including the COVID crash, Luna collapse, FTX bankruptcy, SVB crisis, Bitcoin ETF approval, Trump election rally, Liberation Day tariff shock, and other notable events. For each episode, it produces a four-panel diagnostic figure showing normalised price action, daily return distributions, rolling correlation dynamics across window lengths, and a pre/post correlation shift bar chart. A correlation-regime heatmap summarises all events together, while a scatter-plot grid shows how the co-movement between Bitcoin and the S&P 500 varies by event type. All figures are saved to `outputs/figures/event_*.png` and `outputs/figures/market_events_*.png`, and a summary CSV is written to `outputs/results/market_events_summary.csv`. Embedded HTML cells contain links to the original news sources for each event. This notebook runs last so that it can use the full set of pipeline outputs and serves as a narrative complement to the quantitative results of Chapters 4 and 5.

9. **09_Feature_Ablation.ipynb** — Feature-group ablation. The notebook retrains Ridge, Random Forest, and XGBoost using the nested sequence of feature sets reported in Table 4, starting with a single dependency lag and ending with the full design matrix. The models are evaluated using the walk-forward protocol and the same seeds as in the main experiment. The notebook reproduces the ablation result from Section 4.6 using the frozen dataset without running the full pipeline again.

All notebooks use a shared ROOT-detection routine that locates the project root by searching upward for `config.yaml`. This ensures correct path resolution whether a notebook is launched interactively from within the `notebooks/` directory or executed programmatically through `run_all.py` from the project root.

D.5 Execution flow

The full pipeline is launched through a single entry point that coordinates data acquisition, model training, notebook execution, and document compilation:

Listing 6: Automated full-pipeline runner (`run_all.py`), simplified for readability.

```
import argparse, subprocess, sys, os

BASE = os.path.dirname(os.path.abspath(__file__))
PYTHON = sys.executable

# Step 1: main pipeline (data, models, tables, figures)
subprocess.run([PYTHON, "main.py"], cwd=BASE, check=True)

```

```
# Step 2: execute each notebook with cwd set to project root
for nb in ["01_EDA_Dataset.ipynb", ..., "08_Market_Events_Showcase.ipynb"]:
    subprocess.run(
        [PYTHON, "-m", "jupyter", "nbconvert",
         "--to", "notebook", "--execute", "--inplace",
         f"--ExecutePreprocessor.cwd={BASE}",
         os.path.join(BASE, "notebooks", nb)],
        cwd=BASE
    )

# Step 3: run automated test suite (81 unit and integration tests)
subprocess.run(
    [PYTHON, "-m", "pytest", "tests/", "-v",
     "--junit-xml=outputs/results/test_results.xml"],
    cwd=BASE
)

# Step 4: compile the thesis PDF (test report table auto-included)
subprocess.run(
    ["latexmk", "-pdf", "-interaction=nonstopmode", "main.tex"],
    cwd=os.path.join(BASE, "thesis"), check=True
)
```

At a high level, the execution flow can be summarised as follows:

1. Load configuration and path settings.
2. Refresh raw price data from Yahoo Finance.
3. Compute synchronised log returns.
4. Generate rolling-correlation targets for each asset pair and window.
5. Build lagged and volatility-based feature matrices.
6. Fit all models in walk-forward mode and save predictions.
7. Compute metrics, DM tests, and export tables and figures.
8. Run the investor signal layer and export signal figures.
9. Execute nine Jupyter notebooks to produce supplementary diagnostics, including the market-event showcase (Appendix F).
10. Run the automated `pytest` suite (81 unit and integration tests) and generate the test-report table for Appendix E.
11. Compile the LaTeX thesis document.

The advantage is traceability. Every figure and number in the thesis can be traced back to a single, step-by-step computational procedure. This means that every result can be traced back to its source. This gives confidence in the work, since every number can be linked to how it was obtained. There are no hidden steps or assumptions here. Everything is clear and verifiable. This transparency makes the research more reliable and allows others to verify the work.

D.6 Configuration

Experiment parameters are stored in `config.yaml` and are read once at pipeline startup. This makes it straightforward to change the date range, asset universe, or window set without touching the source code:

Listing 7: Excerpt from `config.yaml` showing key experiment parameters.

```
start_date: "2016-01-01"   # effective start: 2017-11-09 (ETH availability)
end_date:   "2026-05-17"   # frozen window; last trading day in data: 16
    ↪ May 2026

assets:
  crypto:      ["BTC-USD", "ETH-USD"]
  traditional: ["^GSPC", "^IXIC", "GLD", "SLV", "UUP"]

base_asset:           "BTC-USD"
rolling_windows:      [14, 30, 60, 90]
forecast_horizon:     1
use_fisher_transform: true
min_train_size:       800
refit_every:          20
xgb_device:           "cuda"
random_state:         42
```

D.7 Verification performed during the project cleanup

While tightening the project up, several implementation issues were fixed to bring the code to thesis standard.

- To avoid contaminating the entire sample during the preliminary analysis, a leakage-safe DCC benchmark was implemented to prevent look-ahead contamination.
- The feature pipeline was designed so that all relevant models, including DCC, would be evaluated using the same generalisation procedure.
- Feature scaling was added for regularised linear models.
- The XGBoost block became resilient to GPU failures through CPU fallback.
- The investor signal layer was integrated into the main pipeline and exported to dedicated tables and figures.
- Notebook imports and notebook-specific plotting logic were optimised for presentation and interpretation.

These fixes matter because a thesis is judged not only on its idea but on whether the implementation behind it is reliable, reproducible, and internally coherent.

D.8 How to rebuild the thesis artifacts

A practical benefit of the current project state is that the full workflow can be rerun from scratch with a single command from the project root:

Listing 8: Full rebuild from the project root directory.

```
python run_all.py
```

Alternatively, the three stages can be executed individually:

1. Run the main pipeline:
   `python main.py`
   This generates prediction CSVs, metrics, DM-test results, and the majority of figures.
2. Execute the notebooks in order:
   (a) `01_EDA_Dataset.ipynb` — dataset overview and EDA figures.
   (b) `02_GridSearch.ipynb` — cross-validated hyperparameter search.
   (c) `03_Model_Comparison.ipynb` — model-ranking table and heatmaps.
   (d) `04_DM_Tests_Visuals.ipynb` — DM-test heatmap and key forecast figure.
   (e) `05_XGB_vs_DCC.ipynb` — rolling-RMSE, scatter, and error-distribution figures.
   (f) `06_Regime_Analysis.ipynb` — regime catalogue and data-quality diagnostics.
   (g) `07_Robustness_Checks.ipynb` — refit sensitivity, stress threshold sensitivity, bootstrap confidence intervals, signal diagnostics, and cross-asset DM test.
   (h) `08_Market_Events_Showcase.ipynb` — landmark market-event case studies (Appendix F).
   (i) `09_Feature_Ablation.ipynb` — feature-group ablation (Section 4.6).
3. Run the automated test suite and generate the appendix table:

   ```
   python -m pytest tests/ -v --junit-xml=outputs/results/test_results.
       ↪ xml
   ```

4. Rebuild the PDF document from the `thesis/` directory with the same command the runner uses:

   ```
   latexmk -pdf -interaction=nonstopmode main.tex
   ```

   If `latexmk` is not installed, the equivalent manual sequence is `pdflatex main.tex`, then `biber main`, then `pdflatex main.tex` run twice.

Steps 1 and 2 must be completed before step 4 because several figures used in the thesis are produced by the notebooks. Step 3 must be completed before step 4 so that the test-report table (Appendix E) is up to date. This makes the written work maintainable even if the underlying market data are refreshed.

D.9 Final reproducibility note

Reproducibility here is not just a technical nicety. In financial forecasting, where accidental leakage and selective reporting are ever-present temptations, a transparent, rerunnable codebase is part of what makes the empirical conclusions believable.

Appendix E Additional Statistical Tests

This appendix runs three further hypothesis tests to check the robustness of the empirical results. The first is a Mincer-Zarnowitz efficiency test. The second set consists of residual diagnostics, including the Ljung-Box test and the ARCH LM test. The third is the Harvey-Leybourne-Newbold small-sample correction of the Diebold-Mariano statistic.

E.1 Mincer-Zarnowitz Forecast Efficiency Test

The Mincer-Zarnowitz (MZ) test [31] checks whether forecasts are unbiased and informationally efficient. For each model the auxiliary regression

$$y_t = \alpha + \beta\, \hat{y}_t + \varepsilon_t \tag{10}$$

is estimated by OLS and the joint null hypothesis H_0: $\alpha = 0,\ \beta = 1$ is assessed with the conventional OLS F-statistic. Because the realised rolling correlation and forecast errors inherit serial dependence from overlapping windows, the conventional F reference distribution need not hold. The reported values are therefore diagnostic coefficient summaries, not HAC-robust tests of forecast efficiency. A confirmatory MZ analysis should use a heteroskedasticity-and- autocorrelation-consistent Wald test with a lag choice justified by the window length.

Table 9 summarises the percentage of the 24 dependency–window experiments in which each model passes the MZ test, together with its average α and β coefficients.

Table 9: Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments not rejected by the conventional OLS F diagnostic ($p \geq 0.05$) and average coefficient estimates.

Model	Efficient (%)	$\bar{\alpha}$	$\bar{\beta}$	$\bar{F}$
AR(1)	70.8	+0.0006	0.9976	2.0
HAR	70.8	+0.0006	0.9974	2.4
Ridge	45.8	−0.0015	0.9992	4.1
RF	16.7	−0.0019	1.0140	26.2
Naive (last value)	8.3	+0.0097	0.9706	16.3
GBM	8.3	−0.0046	1.0226	35.0
ElasticNet	4.2	−0.0056	1.0175	20.9
Ensemble	0.0	−0.0077	1.0252	30.1
XGBoost	0.0	−0.0252	1.0979	202.2
DCC-GARCH	0.0	−0.0881	1.4433	237.3

AR(1) and HAR are not rejected by the conventional diagnostic at the same rate (70.8% of experiments). Ridge is not rejected in 45.8% of experiments among the regularised ML models. These proportions are descriptive because the reported F-tests are not HAC-robust. DCC-GARCH is rejected by the conventional diagnostic in every experiment; its average $\bar{\beta} = 1.44$ means that realised correlations swing markedly more than its forecasts do. In the regression $y = \alpha + \beta\hat{y}$, a slope above one signals forecasts that are too *compressed*: they systematically under-react to the size of the movements. This is the over-smoothing of DCC-GARCH seen throughout the main results.

Table 10 provides the full detail for the representative pair BTC-USD vs S&P 500 at $w = 30$.

Table 10: Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. Conventional OLS F diagnostic for H_0: $\alpha = 0, \beta = 1$; p_{MZ} is not HAC-robust.

Model	$\hat{\alpha}$	$\hat{\beta}$	F	p_{MZ}	Efficient
Ridge	+0.0071	0.9903	4.76	0.009	No
AR(1)	+0.0071	0.9909	5.07	0.006	No
HAR	+0.0074	0.9913	5.77	0.003	No
Naive (last value)	+0.0114	0.9695	16.74	< 0.001	No
ElasticNet	+0.0065	1.0048	17.18	< 0.001	No
Ensemble	+0.0086	1.0113	39.91	< 0.001	No
RF	+0.0221	0.9951	57.04	< 0.001	No
GBM	+0.0226	0.9945	62.30	< 0.001	No
DCC-GARCH	−0.0886	1.4616	261.03	< 0.001	No
XGBoost	+0.0436	1.0316	318.18	< 0.001	No

At this particular pair and window all models are rejected by the conventional OLS diagnostic, but the coefficient gradient is steep. Ridge ($F = 4.76$), AR(1) ($F = 5.07$), and HAR ($F = 5.77$) have coefficients close to $(0, 1)$ under this descriptive regression. The DCC-GARCH statistic (261.0) and XGBoost statistic (318.2) are much larger, but neither difference is treated as formal evidence of misspecification without HAC-robust inference.

E.2 Residual Diagnostics: Ljung-Box and ARCH LM Tests

Two classical residual tests are applied to the OOS forecast errors $e_t = y_t - \hat{y}_t$ for every model × dependency × window combination (240 cases in total).

Ljung-Box Q-test (10 lags) [32]: tests H_0: no serial autocorrelation. Rejection indicates that the model has not fully captured the temporal structure of the target series.

ARCH LM test (5 lags) [33]: regresses e_t^2 on its own lags and tests H_0: no conditional heteroscedasticity. Rejection signals remaining volatility clustering in the residuals.

Table 11 reports, for each model, the percentage of experiments in which the residuals pass each test at the 5% level.

Table 11: Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$.

Model	No autocorr (%)	No ARCH (%)
Naive (last value)	50.0	58.3
AR(1)	41.7	62.5
HAR	41.7	58.3
Ridge	20.8	54.2
ElasticNet	0.0	37.5
Ensemble	0.0	12.5
RF	0.0	0.0
GBM	0.0	0.0
XGBoost	0.0	0.0
DCC-GARCH	0.0	0.0

The autocorrelation results show a clear pattern. The Naive (last-value) model and the autoregressive models (AR(1), HAR) capture part of the serial structure and pass the Ljung-Box test in 42–50% of experiments. Ridge passes in 21% of cases, showing that it picks up the autoregressive signal from the lag features. ElasticNet is a particular case: it fails every autocorrelation test (0%) yet still has a meaningful ARCH pass rate (38%), so it removes some temporal dependence but not all. The tree-based methods (RF, GBM, XGBoost) and DCC-GARCH retain significant autocorrelation and heteroscedasticity in every single experiment: they never encode the target's autoregressive structure, so that structure leaks into the residuals, which remain far from white noise. The large Q-statistics for the tree ensembles (for example, $Q > 1{,}000$ for XGBoost on most pairs) make this clear: these forecasters simply do not carry the AR structure forward from training.

E.3 Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test

The standard Diebold-Mariano statistic uses the asymptotic standard-normal distribution, which tends to over-reject in finite samples. Harvey, Leybourne & Newbold [34] propose multiplying the DM statistic by the correction factor

$$k = \sqrt{\frac{n + 1 - 2h + h(h-1)/n}{n}}, \tag{11}$$

where n is the number of forecast observations and h the forecast horizon, and comparing the result to a $t(n-1)$ distribution. For $h = 1$ the factor simplifies to $k = \sqrt{(n-1)/n}$, which is slightly below one and therefore shrinks the statistic; with $n > 1{,}000$ the shrinkage is of the order of 10^{-4}. This correction addresses finite-sample size, not the serial dependence in loss differentials induced by overlapping rolling windows.

Applied to all 336 DM comparisons (168 ML models vs DCC-GARCH and 168 vs Naive (last value)), Table 12 shows that the HLN correction changes the reported zero-lag statistics only trivially (average correction factor $k = 0.999774$). This establishes only that the displayed calculations are insensitive to the finite-sample HLN multiplier. It does

not validate their zero-lag variance estimator or turn them into formal significance tests for the overlapping rolling target.

Table 12: HLN-corrected DM diagnostic: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); $\mathrm{DM}_{\mathrm{HLN}}$: corrected statistic ($t_{n-1}$). Reported p-values retain the zero-lag variance assumption and are descriptive.

Model	DM	p_{DM}	$\mathrm{DM}_{\mathrm{HLN}}$	p_{HLN}	Sig.
Ridge	26.86	< 0.001	26.86	< 0.001	***
HAR	26.83	< 0.001	26.83	< 0.001	***
ElasticNet	26.97	< 0.001	26.97	< 0.001	***
Ensemble	27.05	< 0.001	27.04	< 0.001	***
RF	26.44	< 0.001	26.43	< 0.001	***
GBM	26.16	< 0.001	26.16	< 0.001	***
XGBoost	24.29	< 0.001	24.28	< 0.001	***

Two aspects of the comparison set in Table 12 should be made explicit. First, the seven *fitted* models are compared against two benchmarks: the DCC-GARCH econometric model and the naive last-value baseline. AR(1) is treated as one of the persistence baselines rather than as a test model, so it does not appear as a separate row; its position relative to the fitted models is established instead by the average-RMSE ranking of Table 3, where AR(1), HAR, and Ridge have closely clustered errors. Second, the Diebold–Mariano test assumes non-nested forecasts, whereas every fitted model here includes the lagged target among its predictors and therefore nests the persistence baselines. The statistics against Naive (last value), and by extension any test against AR(1), should accordingly be read as descriptive; the conclusions rest on the comparison the test is designed for, namely the fitted models against the non-nested DCC-GARCH benchmark.

E.4 Automated Validation Suite

In addition to the statistical tests above, the full project ships with an automated `pytest` suite of **81 unit and integration tests** organised into ten test classes. These are predominantly consistency and regression checks: they verify that a fresh run reproduces the published outputs (file presence, value ranges, sign conventions, and key relationships such as the average ranking of AR(1) against DCC-GARCH); they do not prove the methods correct in an absolute sense. The suite is executed automatically as part of the reproducibility runner (`python run_all.py`) between the notebook step and the final LaTeX compilation, so every rebuilt PDF contains a fresh test report.

The test classes cover the following aspects of the pipeline:

- **Dataset Integrity**. Price and return files exist. They contain the correct tickers, positive prices, sorted and non-duplicated dates, and extend to at least 2026.
- **Data Quality**. Missing-rate thresholds are met, zero-price detection works, annualised volatility is reasonable, and pairwise coverage overlap is adequate.
- **Feature Engineering**. The Fisher-z round-trip identity holds, rolling correlation falls within the expected range, and prediction CSVs contain no look-ahead information.

- **Model Metrics**. RMSE is greater than zero, R^2 is less than or equal to one, MAE is less than or equal to RMSE, all windows and expected models are present, and AR(1) performs better than DCC-GARCH on average.
- **DM Tests**. DM statistics are finite, p-values lie between zero and one, and the sign convention for Ridge versus DCC-GARCH is correct.
- **Signal Metrics**. Balanced accuracy and F1 score are between zero and one, AUC is above random, and the exit rate falls within a sensible range.
- **Walk-Forward**. The correct number of prediction CSVs are produced, predictions are non-trivial, and indices align with returns.
- **Output Files**. After a full pipeline run, all expected CSV, JSON, PNG, and `.tex` artefacts are present.
- **Bootstrap CI**. The lower bound is less than or equal to the mean, which is less than or equal to the upper bound for RMSE and R^2, and CI width is below 0.10.
- **Sensitivity**. All five `refit_every` values are present. RMSE lies within a physically plausible range.

Table 13 summarises the suite by test class — the number of tests, the pass rate, and the total execution time for each. The complete per-test log, with parametrised cases and individual timings, is stored in `outputs/results/test_results.xml` in the repository. All 81 tests pass on the present outputs, with none skipped.

Table 13: Automated validation suite, summarised by test class. All 81 tests pass on the present outputs, with none skipped. The complete per-test log, including parametrised cases and per-test timings, is written to `outputs/results/test_results.xml` in the repository.

Test class	Tests	Passed	Time (s)
`TestDatasetIntegrity`	13	13/13	0.12
`TestDataQuality`	9	9/9	0.09
`TestFeatureEngineering`	6	6/6	0.05
`TestModelMetrics`	12	12/12	0.03
`TestDMTests`	8	8/8	0.02
`TestSignalMetrics`	7	7/7	0.02
`TestWalkForward`	5	5/5	0.07
`TestOutputFiles`	14	14/14	0.00
`TestBootstrapCI`	3	3/3	0.00
`TestSensitivity`	4	4/4	0.00
Total	**81**	**81**	**0.41**

Appendix F Market-Event Case Studies

This appendix examines key market events from 2020 to 2026 using the thesis's main variable: the rolling Pearson correlation between Bitcoin and traditional assets. The main timeline and correlation regime map first summarise all eleven events. This is followed by a detailed analysis of four representative events: the COVID crash, the FTX collapse, the SVB banking crisis, and the 2025 "Liberation Day" tariff shock. These events were selected to cover the main types of regimes: a liquidity-driven co-crash, a crypto-specific shock, a brief safe-haven decoupling, and a macro risk-off shock. The remaining seven events are marked on the main timeline and are fully reproduced in the 08_Market_Events_Showcase.ipynb notebook in the repository. The goal is to link the quantitative results from Chapter 4 to specific historical events and to show how the regime-transition problem appears in practice.

Master Timeline

Figure 27 shows the full sample period with all eleven events marked. The three panels display Bitcoin's normalised price trajectory on a logarithmic scale, the S&P 500 and NASDAQ together with gold, and the 30-day rolling Pearson correlation between BTC and the S&P 500. The event markers show how correlation spikes cluster around macro risk-off episodes while remaining subdued during crypto-specific events.

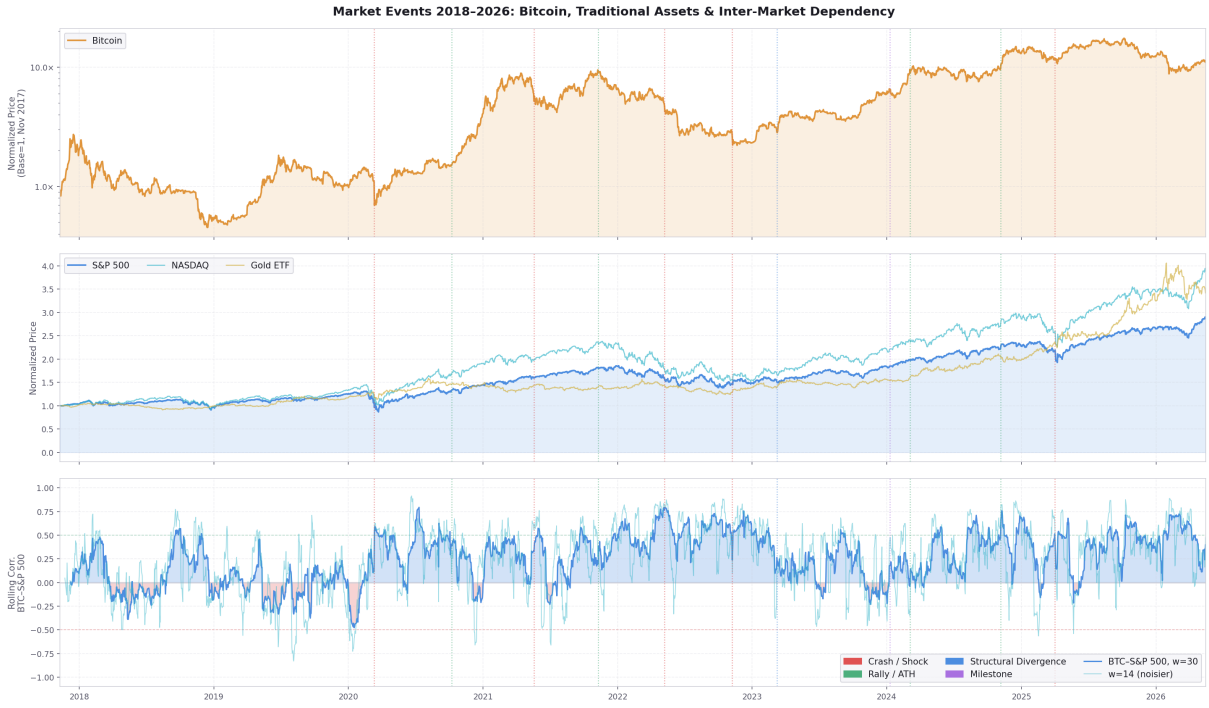


Figure 27: Master timeline 2018–2026: Bitcoin price, conventional-asset prices, and the 30-day rolling BTC–S&P 500 correlation, with all eleven landmark events marked. Red dotted lines = crash/shock events; green = rallies and all-time highs; blue = structural divergences; purple = institutional milestones.

Correlation Regime Map

Figure 28 summarises all events jointly. The left panel shows the 30-day average BTC correlation with each conventional asset *after* each event; the right panel shows the change in correlation ($\Delta\rho = \text{post} - \text{pre}$), isolating the regime shift associated with the event itself.

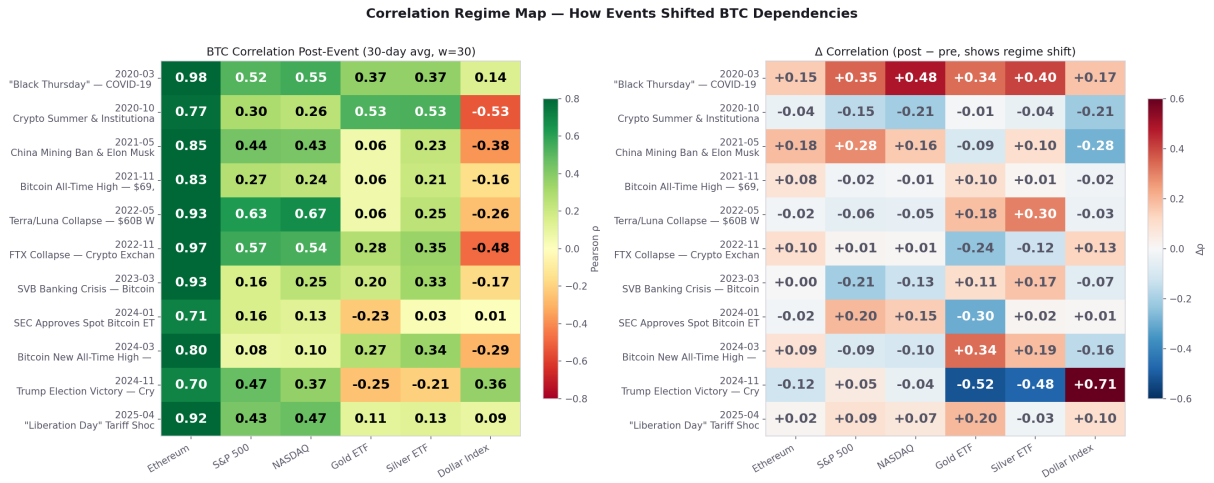


Figure 28: Correlation regime map. Left: post-event BTC correlation (30-day average) with each conventional asset. Right: change in correlation relative to the 30-day pre-event average. Red/blue = strong positive/negative shifts.

Individual Event Panels

The following figures provide a four-panel deep-dive for each of the four representative events: (top-left) normalised price trajectories for Bitcoin and the most relevant comparison assets; (top-right) daily log-returns as a bar chart; (bottom-left) 30-day and 14-day rolling BTC correlation with the primary comparison asset; (bottom-right) pre-/post-event correlation for all pairs.

Event 1 — COVID Black Thursday (12 March 2020). Bitcoin fell about 37% in a single session, sharply outpacing the S&P 500's 10% drop. The 30-day BTC–S&P 500 correlation rose from about 0.17 to about 0.52. This was a liquidity-driven sell-off, in which Bitcoin's diversification benefit temporarily disappeared: with margin calls forcing simultaneous liquidation across asset classes, the usual distinctions between them stopped mattering. Figure 29 shows the episode.

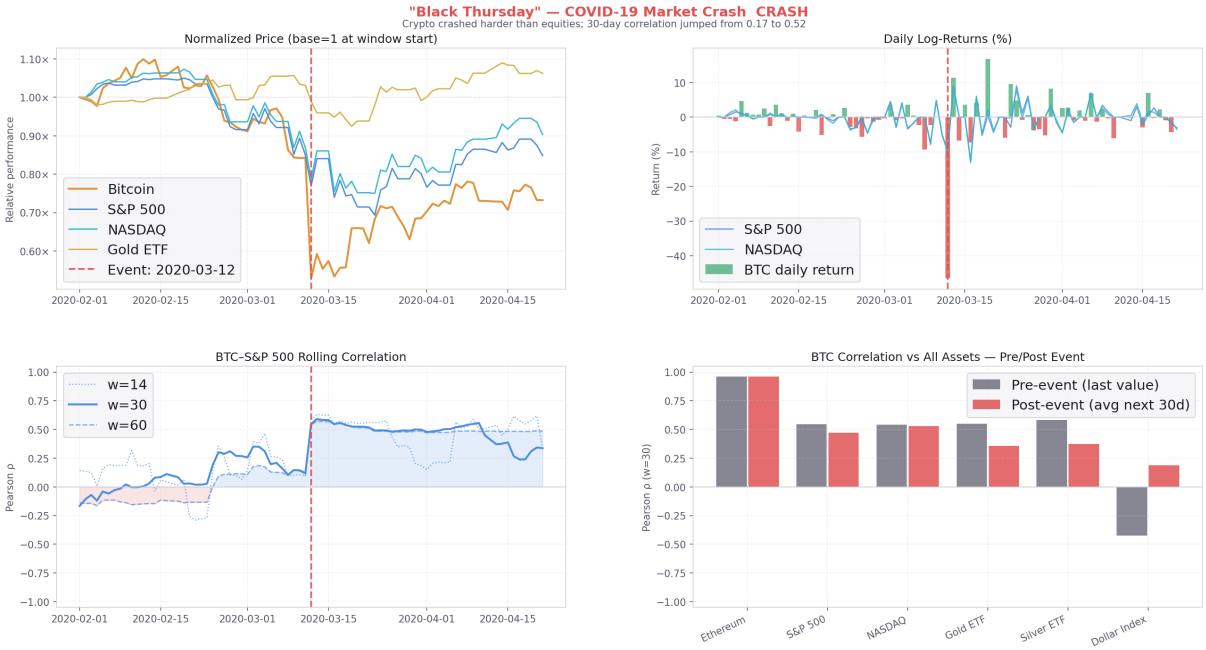


Figure 29: Event 1 — COVID Black Thursday, 12 March 2020. Source: Reuters.

Event 2 — FTX Collapse (8–11 November 2022). The exchange filed for bankruptcy and Bitcoin fell about 25% over the following two weeks. The S&P 500 *rose* that same week on a positive CPI print, producing a sharp short-term decoupling: the 14-day BTC–equity correlation fell from about 0.6 toward zero, while the 30-day measure barely moved (about 0.55). The contrast between the two window lengths is the point of the episode. A shock confined to the crypto market registers immediately in the short window and is almost invisible in the longer one, which is exactly the trade-off between responsiveness and smoothness discussed in Section 4.12. Figure 30 shows the episode.

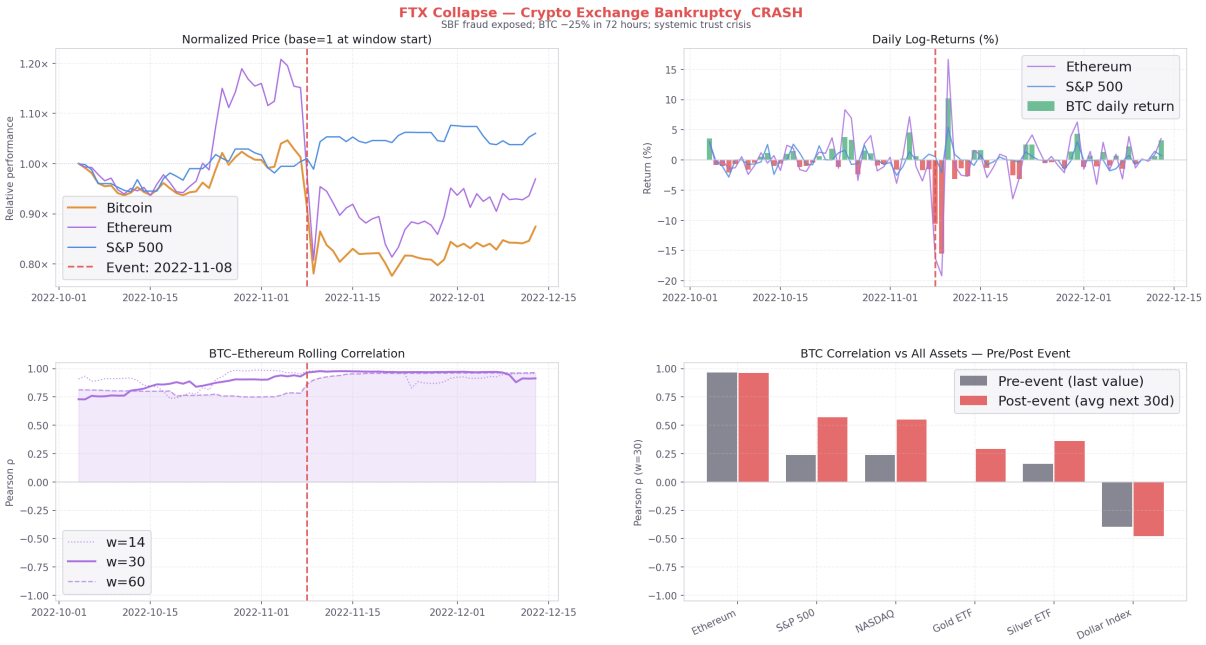


Figure 30: Event 2 — FTX collapse, 8–11 November 2022. Source: BBC News.

Event 3 — SVB Banking Crisis (10 March 2023). Silicon Valley Bank's failure triggered a Bitcoin rally of about 40% over the following week as investors sought alternatives to the banking system. Both Bitcoin and gold rose (gold about 8%) while equities fell. This is one of the rare episodes in the sample where the short-window correlation turned negative: the 14-day BTC–S&P 500 measure dipped to about −0.4, while the 30-day measure fell from 0.37 to 0.16. It is also the only one of the four events in which Bitcoin behaved as the safe-haven asset rather than the risk asset. Figure 31 shows the episode.

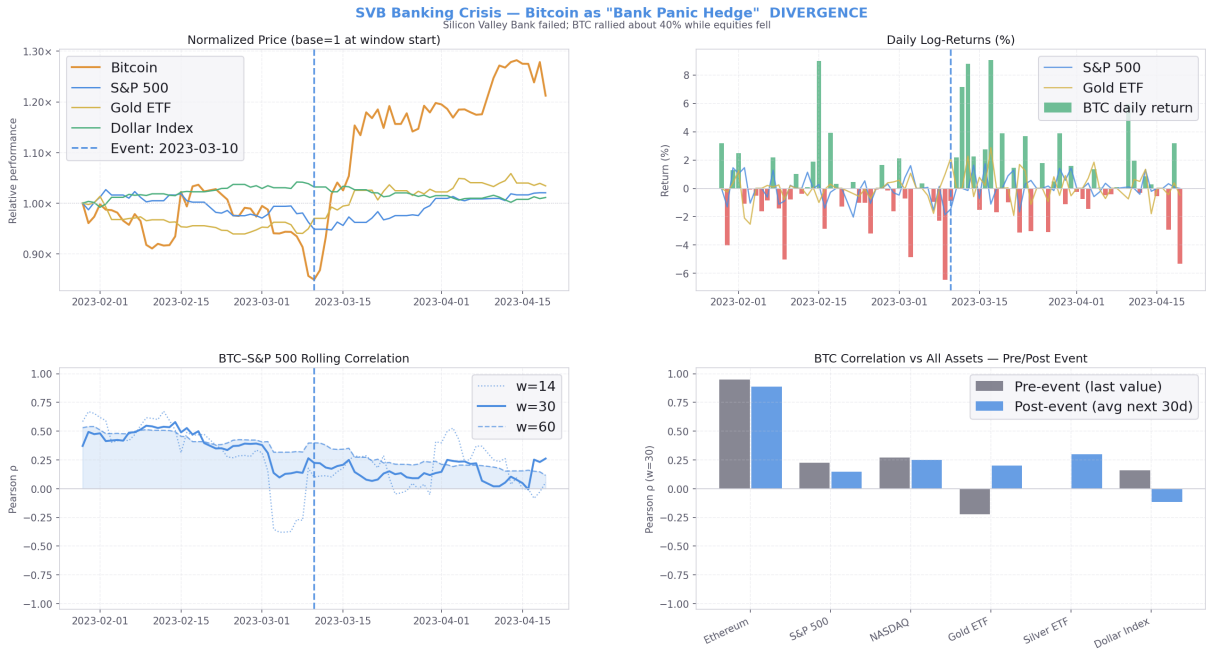


Figure 31: Event 3 — SVB banking crisis, 10 March 2023. Source: The New York Times.

Event 4 — "Liberation Day" Tariff Shock (2 April 2025). Sweeping tariffs on 185 countries caused the S&P 500 to fall about 12% in four trading days, its fastest drop since March 2020. Bitcoin fell alongside equities over the same week, while gold climbed to record highs above $3 000/oz. Both risk assets sold off together, confirming that Bitcoin behaves as a risk asset rather than a safe haven during acute macro shocks; even so, the 30-day rolling BTC–S&P 500 correlation showed no clear spike, edging only from about 0.33 to 0.43, because the sharp partial recovery in the days that followed desynchronised the daily moves. The episode illustrates that even a major macro shock need not durably raise the rolling dependency, which is the main reason a correlation-based warning layer cannot be expected to fire reliably on macro events. Figure 32 shows the episode.

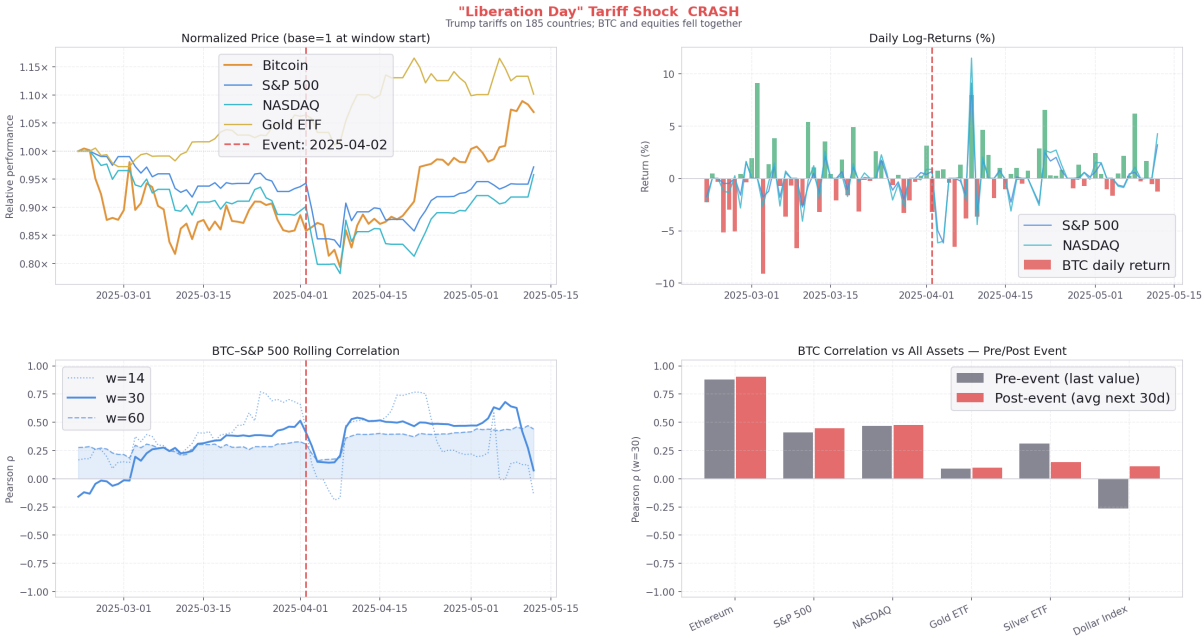


Figure 32: Event 4 — "Liberation Day" tariff shock, 2 April 2025. Source: Reuters.

Interpretation

Across the set of episodes described above and others highlighted in the main timeline, four key findings complement the overall quantitative results obtained in Chapter 4.

1. Macro risk-off events lead to the largest spikes in correlation. The COVID crash in 2020 and the Fed's tightening cycle from 2021 to 2022 led to the BTC–equity correlation reaching the highest sustained levels in the sample. The 30-day indicator rose to about 0.52 in March 2020 and remained at 0.6–0.7 during the 2022 selloff. Figure 4 also shows the periods when the rolling RMSE reaches its maximum. In contrast, the shock caused by Liberation Day in 2025 was accompanied by a sharp drop in prices, but the sliding correlation remained virtually unchanged. This shows that even severe macroeconomic shocks do not always increase the measurable dependence.

2. Due to the shocks associated with cryptocurrencies, equity correlations often remain virtually unchanged. The ban on mining in China, the collapse of Luna and the bankruptcy of FTX led to large losses of bitcoin, while the stock markets remained virtually unchanged. These events led to only small and temporary changes in the dynamic correlation and did not lead to long-term shifts in the relationship.

3. The institutional era that has come since 2024 has led to increased correlation. After the approval of the ETF in January 2024, institutional capital began to flow into bitcoin through the ETF products. This means that bitcoin is increasingly trading under the same risk-on/risk-off frameworks as growth equities. Thus, the 30-day BTC–NASDAQ correlation remains at a high level, averaging about 0.36 from 2024 to 2026. However, it has not yet reached the peak of 0.58, which was observed during the monetary policy tightening cycle in 2022, when both assets were sold off at the same time.

4. Safe-haven episodes do happen, but they are short-lived. The SVB crisis led to a

genuinely negative correlation between BTC and equity, but it lasted only two to three weeks before it returned to normal. Such a short duration helps explain why persistence-based models continue to dominate out-of-sample: the autoregression signal is strong enough to absorb regime shifts after a short delay, even if it cannot anticipate them.

A complete set of event level data, summary statistics, and news links for all eleven episodes are automatically generated by notebook 08_Market_Events_Showcase.ipynb, which is launched as the final step of `run_all.py`.

References

[1] François Longin and Bruno Solnik, „Extreme Correlation of International Equity Markets", in: *The Journal of Finance* 56.2 (2001), pp. 649–676.

[2] Satoshi Nakamoto, „Bitcoin: A peer-to-peer electronic cash system", in: *Decentralized Business Review* (2008).

[3] Elie Bouri et al., „On the hedge and safe haven properties of Bitcoin: Is it really more than a diversifier?", in: *Finance Research Letters* 20 (2017), pp. 192–198.

[4] Shaen Corbet et al., „Exploring the dynamic relationships between cryptocurrencies and other financial assets", in: *Economics Letters* 165 (2018), pp. 28–34.

[5] Qiang Ji et al., „Network causality structures among Bitcoin and other financial assets", in: *The Quarterly Review of Economics and Finance* 70 (2018), pp. 203–213.

[6] Ladislav Kristoufek, „What are Bitcoin's main drivers?", in: *PLOS ONE* 10.4 (2015), e0123923.

[7] Anne Haubo Dyhrberg, „Bitcoin, gold and the dollar – A GARCH volatility analysis", in: *Finance Research Letters* 16 (2016), pp. 85–92.

[8] Tony Klein, Hien Pham Thu, and Thomas Walther, „Bitcoin is not the new gold – A comparison of volatility, correlation, and portfolio performance", in: *International Review of Financial Analysis* 59 (2018), pp. 105–116.

[9] Andrew J. Patton, „Modelling asymmetric exchange rate dependence", in: *International Economic Review* 47.2 (2006), pp. 527–556.

[10] Robert F. Engle and Kevin Sheppard, „Theoretical and empirical properties of dynamic conditional correlation multivariate GARCH", in: *NBER Working Paper* 8554 (2001).

[11] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models", in: *Journal of Business & Economic Statistics* 20.3 (2002), pp. 339–350.

[12] Shihao Gu, Bryan Kelly, and Dacheng Xiu, „Empirical asset pricing via machine learning", in: *The Review of Financial Studies* 33.5 (2020), pp. 2223–2273.

[13] Fulvio Corsi, „A simple approximate long-memory model of realized volatility", in: *Journal of Financial Econometrics* 7.2 (2009), pp. 174–196.

[14] Torben G. Andersen et al., „Modeling and forecasting realized volatility", in: *Econometrica* 71.2 (2003), pp. 579–625.

[15] Hui Zou and Trevor Hastie, „Regularization and Variable Selection via the Elastic Net", in: *Journal of the Royal Statistical Society: Series B* 67.2 (2005), pp. 301–320.

[16] Leo Breiman, „Random Forests", in: *Machine Learning* 45.1 (2001), pp. 5–32.

[17] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine", in: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[18] Tianqi Chen and Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System", in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[19] Francis X. Diebold and Roberto S. Mariano, „Comparing Predictive Accuracy", in: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

[20] yfinance developers, *yfinance Documentation*, 2026, `https://ranaroussi.github.io/yfinance/` (visited on 03/10/2026).

[21] Todd E. Clark and Kenneth D. West, „Approximately Normal Tests for Equal Predictive Accuracy in Nested Models", in: *Journal of Econometrics* 138.1 (2007), pp. 291–311.

[22] Raffaella Giacomini and Halbert White, „Tests of Conditional Predictive Ability", in: *Econometrica* 74.6 (2006), pp. 1545–1578.

[23] Marcos Lopez de Prado, *Advances in Financial Machine Learning*, Wiley, 2018.

[24] John C. Platt, „Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods", in: *Advances in Large Margin Classifiers*, ed. by Alexander J. Smola et al., MIT Press, 1999, pp. 61–74.

[25] Alexandru Niculescu-Mizil and Rich Caruana, „Predicting Good Probabilities with Supervised Learning", in: *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632.

[26] Ivo Welch and Amit Goyal, „A Comprehensive Look at the Empirical Performance of Equity Premium Prediction", in: *The Review of Financial Studies* 21.4 (2008), pp. 1455–1508.

[27] scikit-learn developers, *scikit-learn User Guide*, 2026, `https://scikit-learn.org/stable/user_guide.html` (visited on 03/10/2026).

[28] XGBoost developers, *XGBoost Documentation*, 2026, `https://xgboost.readthedocs.io/en/stable/` (visited on 03/10/2026).

[29] arch developers, *arch Documentation*, 2026, `https://bashtage.github.io/arch/` (visited on 03/10/2026).

[30] elektrotehnika, *finlatex: finthesis dokument klasa*, 2026, `https://github.com/elektrotehnika/finlatex` (visited on 03/10/2026).

[31] Jacob Mincer and Victor Zarnowitz, „The Evaluation of Economic Forecasts", in: *Economic Forecasts and Expectations*, ed. by Jacob Mincer, National Bureau of Economic Research, 1969, pp. 1–46.

[32] Greta M. Ljung and George E. P. Box, „On a measure of lack of fit in time series models", in: *Biometrika* 65.2 (1978), pp. 297–303.

[33] Robert F. Engle, „Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation", in: *Econometrica* 50.4 (1982), pp. 987–1007.

[34] David Harvey, Stephen Leybourne, and Paul Newbold, „Testing the equality of prediction mean squared errors", in: *International Journal of Forecasting* 13.2 (1997), pp. 281–291.